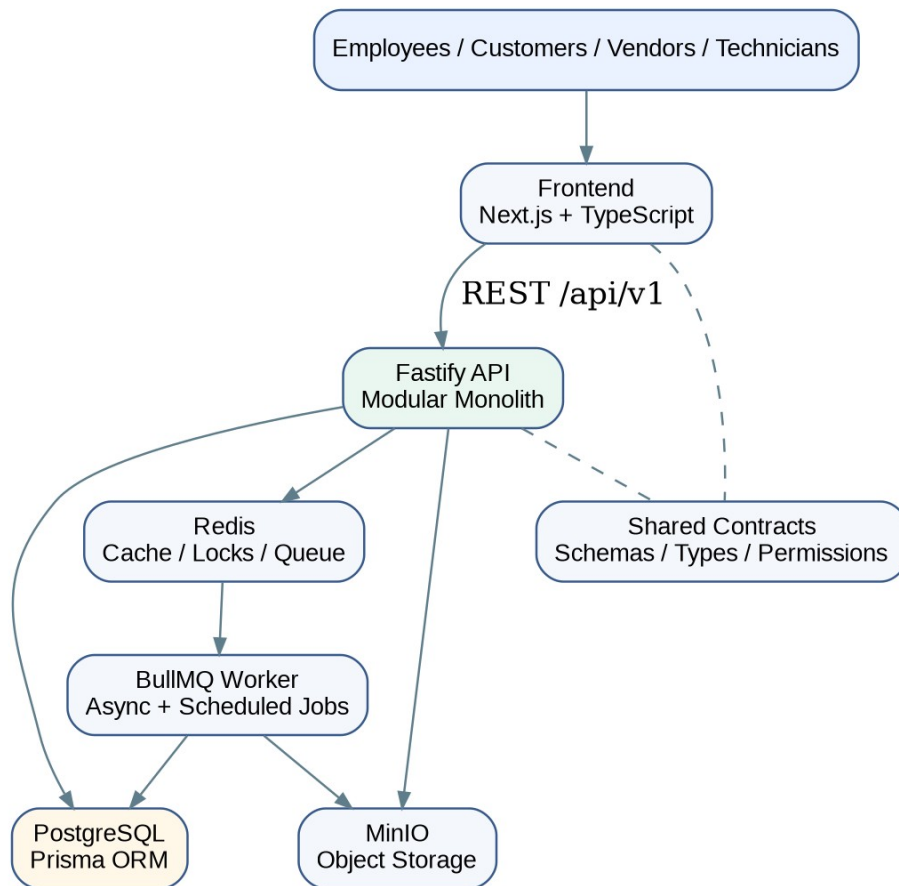


NEXORA ERP

Complete Software Architecture, Database, API & Implementation Specification

Integrated Enterprise Operations, Asset, Project & Service Management Platform



Reference technology architecture

Architecture baseline: TypeScript monorepo | Fastify modular monolith | PostgreSQL + Prisma | MinIO | Redis + BullMQ

Document version 1.0 | 28 August 2026

Document Control

Item	Specification
Document	NEXORA ERP Complete Technical Specification
Version	1.0
Architecture style	Opinionated domain-first modular monolith
Primary backend	Fastify + TypeScript
Database	PostgreSQL
ORM	Prisma ORM
Object storage	MinIO (S3-compatible)
Async processing	Redis + BullMQ
Workspace	Single TypeScript monorepo with root workspace dependency installation
AI usage	None required for application functionality
Status	Architecture and implementation baseline

Authority of this document

This specification consolidates the functional NEXORA ERP concept with the final technology and architecture decisions. Where an earlier draft suggested alternative technologies, this document supersedes those choices with the Fastify/TypeScript/PostgreSQL/Prisma/MinIO stack defined here.

Contents

- 1. Executive Overview and Business Scope
- 2. Functional Specification (complete project scope)
- 3. Final Technology Stack
- 4. Monorepo and Opinionated Modular-Monolith Architecture
- 5. Cross-Module Rules and Transaction Boundaries
- 6. Database Design Standards
- 7. Database Entity Catalog and Relationship Mapping
- 8. API Architecture and Conventions
- 9. API Endpoint Catalog
- 10. API Contracts and Example Payloads
- 11. Authentication, Sessions, RBAC and Tenant Isolation
- 12. MinIO File Storage Design
- 13. Redis, BullMQ and Eventing
- 14. Security Architecture
- 15. Logging, Audit, Observability and Reporting
- 16. Testing Strategy
- 17. Docker, Deployment, CI/CD and Infrastructure
- 18. Development Roadmap and Definition of Done
- Appendices: permissions, events, status models and engineering rules

1. Executive Overview and Business Scope

NEXORA ERP is designed as a commercial-grade enterprise operations platform for engineering, field-service and asset-driven organizations. Its defining characteristic is lifecycle continuity: customer demand becomes a contract and project; project demand drives procurement and inventory; inventory becomes installed assets; assets generate maintenance and service work; all operational activity feeds finance, audit, reporting and profitability.



End-to-end business lifecycle controlled by NEXORA ERP

Design objective	Interpretation
Connected business operations	Avoid isolated CRUD modules. Each domain produces records consumed by downstream domains.
Multi-tenant SaaS readiness	All tenant-owned records are scoped by organizationId and authorization context.
Transactional integrity	Stock, finance, approval and receiving workflows must be atomic.
Traceability	Every important object has status history, audit events, references and ownership.
Configurable enterprise controls	Approvals, permissions, custom fields, statuses, rules and workflows are data-driven where practical.
No AI dependency	Rules, queues, schedules, calculations and deterministic search provide automation.
Modular growth	One deployable application with hard module boundaries so domains can be extracted later only if justified.

2. Functional Specification

The following functional scope is retained as the product baseline. It defines the business capabilities that the technical architecture must support. The legacy technology-choice sections are intentionally replaced later by the final stack and implementation architecture.

NEXORA ERP

Integrated Enterprise Operations, Asset, Project & Service Management Platform

1. Project Overview

NEXORA ERP is a web-based enterprise resource planning system designed for organizations that manage physical products, customer projects, employees, suppliers, warehouses, equipment, installations, maintenance operations, financial transactions, and field-service teams.

The purpose of the system is not simply to provide separate HR, inventory, finance, and project modules.

The main goal is to create a single connected business environment where an activity performed in one department automatically becomes relevant to other departments.

For example, when a company receives a new customer project, that project may require:

- employees,
- equipment,
- inventory,
- purchasing,
- transportation,
- approvals,
- contractors,
- customer documentation,
- installations,
- invoicing,
- payments,
- warranty tracking,
- preventive maintenance,

- field-service visits,
- and profitability calculations.

NEXORA ERP manages this entire lifecycle.

The original concept follows the complete path from customer and contract through procurement, warehouse operations, installation, asset registration, invoicing, warranty, maintenance, and service history.

2. Target Organizations

NEXORA ERP can be used by organizations such as:

- CCTV and security-system companies
- Solar installation companies
- HVAC companies
- Networking companies
- IT service providers
- Telecom contractors
- Generator and UPS companies
- Fire alarm companies
- Electrical contractors
- Facilities-management companies
- Industrial equipment companies
- Building maintenance companies
- Engineering contractors
- Construction subcontractors
- Managed service providers
- Data-centre maintenance companies
- Access-control companies
- Smart-building companies

Instead of creating separate ERP versions for every industry, NEXORA should allow organizations to configure their own:

- departments,
- workflows,
- approval levels,
- warehouses,
- project types,
- asset categories,
- ticket categories,
- employee roles,
- financial rules,
- document types,
- service types,
- and business policies.

This makes the ERP reusable across industries.

3. Core Business Scenario

Consider a company called:

SecureTech Solutions

A university requests:

Business example

Installation of 100 CCTV cameras, 10 NVRs and complete networking infrastructure across five buildings.

The project starts with an inquiry.

The complete workflow becomes:

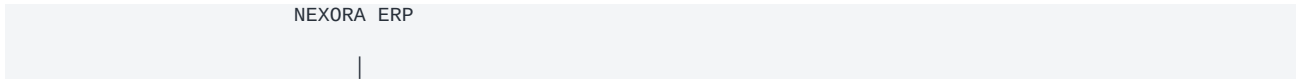
```
Customer Inquiry
  ↓
Opportunity
  ↓
Site Survey
  ↓
Quotation
```

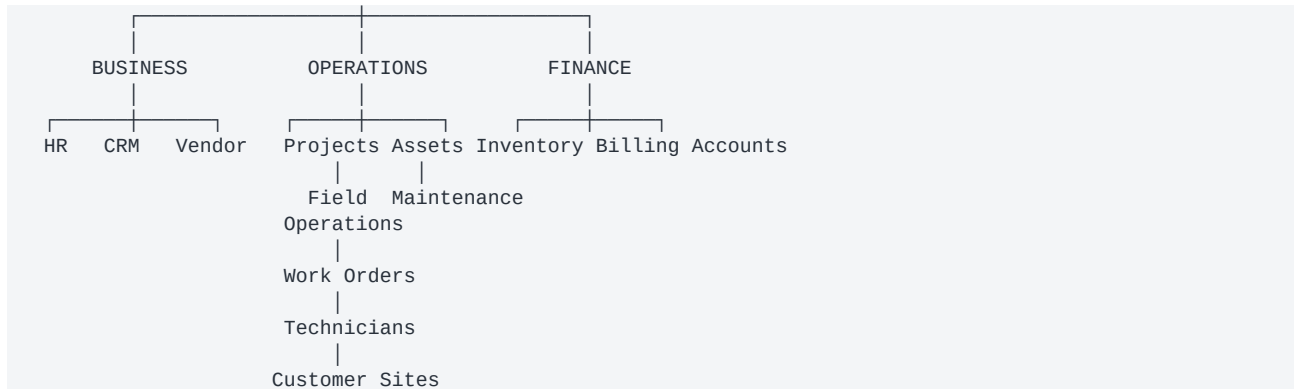


This is the central concept of NEXORA ERP.

4. System Architecture Concept

NEXORA should operate as a modular ERP.





All modules use one shared database and one common authorization framework.

5. Multi-Company and Multi-Branch Architecture

This feature would significantly improve the professional quality of the project.

The ERP should support multiple companies.

For example:

```

NEXORA ERP

Tenant 1
SecureTech Solutions

Tenant 2
PowerGrid Engineering

Tenant 3
Vision Security Systems
    
```

Each organization should have completely separated information.

Inside one organization:

```

SecureTech Solutions

├── Head Office
│   └── Lahore
├── Islamabad Branch
├── Karachi Branch
└── Multan Branch
    
```

Every branch may have:

- separate employees,
- warehouses,
- expenses,
- customers,
- projects,
- stock,
- cash accounts,
- managers,
- and financial reporting.

Head-office users can view consolidated information.

6. Organization Management Module

This module establishes the company structure.

The administrator should be able to configure:

Company

- Company name
- Registration number
- Tax number
- Logo
- Address
- Contact details
- Currency
- Financial year
- Time zone
- Default language

Branches

Each company can contain multiple branches.

Example:

```

Company
  ↓
Branch
  ↓
Department
  ↓
Team
  ↓
Employee
    
```

Departments

Examples:

- Management
- HR
- Finance
- Procurement
- Sales
- Operations
- Engineering
- Warehouse
- Maintenance
- IT
- Customer Support

7. Identity and Access Management

Security must be treated as a core ERP capability.

Users should authenticate through:

```

Email / Employee ID
  +
Password
  +
Optional MFA
    
```

Possible account states:

```

ACTIVE
LOCKED
SUSPENDED
PASSWORD_EXPIRED
DISABLED
    
```

The system should support:

- Password policies
- Login attempt limits
- Session management
- Device/session history

- Password reset
- Multi-factor authentication
- Account lockout
- Forced logout
- Session expiration

8. Role-Based Access Control

Instead of simple Admin/User permissions, implement granular RBAC.

Example roles:

```
Super Administrator
Director
General Manager
Branch Manager
Finance Manager
HR Manager
Procurement Manager
Sales Manager
Project Manager
Warehouse Manager
Maintenance Manager
Technician
Accountant
Sales Officer
Customer
Vendor
Auditor
```

Permissions can include:

```
project.view
project.create
project.update
project.delete

purchase_request.create
purchase_request.review
purchase_request.approve

purchase_order.create
purchase_order.approve

inventory.view
inventory.receive
inventory.issue
inventory.transfer
inventory.adjust

invoice.create
invoice.approve
invoice.cancel

employee.view
employee.manage

workorder.create
workorder.assign
workorder.close
```

Roles receive permissions.

Users receive roles.

This matches the original goal of implementing proper granular permissions rather than one global administrator role.

9. Employee and HR Management

HR should be more than employee CRUD.

Employee profile

Each employee can contain:

- Employee ID
- Name
- Photograph
- Department
- Branch
- Job title
- Manager
- Joining date
- Employment type
- Salary
- Contact information
- Emergency contact
- Bank information
- Documents
- Certifications
- Skills
- Employment status

Example:

```
EMP-00218

Name:
Ahmed Khan

Designation:
Network Engineer

Department:
Operations

Branch:
Lahore

Manager:
Project Manager

Status:
ACTIVE
```

10. Attendance Management

Possible attendance methods:

- Manual attendance
- Check-in/check-out
- QR attendance
- Mobile attendance
- Attendance import

Statuses:

```
Present
Absent
Late
Half Day
Remote
On Site
Leave
Holiday
```

Reports:

- Daily attendance
- Monthly attendance
- Late employees
- Employee attendance percentage

- Overtime

11. Leave Management

Employee:

```

Leave Request
  ↓
Manager Review
  ↓
HR Review
  ↓
Approved / Rejected
    
```

Leave types:

- Annual leave
- Sick leave
- Casual leave
- Unpaid leave
- Emergency leave

The system maintains leave balances automatically.

12. Payroll

Payroll can include:

```

Basic Salary
+ Allowances
+ Overtime
+ Bonus
- Tax
- Loan Deduction
- Absence Deduction
-----
Net Salary
    
```

Payroll status:

```

DRAFT
CALCULATED
REVIEWED
APPROVED
PAID
    
```

Payslips can be generated as PDFs.

13. Customer Management

Although NEXORA is primarily an ERP, it should contain lightweight CRM functionality.

Customer profiles may contain:

- Company name
- Contact person
- Phone number
- Email
- Billing address
- Service locations
- Tax information
- Contracts
- Projects
- Assets
- Tickets
- Invoices
- Payments
- Documents

A customer becomes connected to everything else.

```

Customer
├── Contacts
    
```

```

├── Sites
├── Projects
├── Contracts
├── Assets
├── Tickets
├── Work Orders
├── Invoices
└── Payments
    
```

14. Sales and Opportunity Management

This can extend the ERP further.

Sales flow:

```

Lead
↓
Qualified Lead
↓
Opportunity
↓
Site Survey
↓
Quotation
↓
Negotiation
↓
Customer Approval
↓
Sales Order
↓
Contract
↓
Project
    
```

Opportunity information can include:

- Customer
- Estimated value
- Probability
- Expected closing date
- Assigned sales officer
- Products/services
- Notes
- Attachments
- Sales stage

Stages:

```

NEW
CONTACTED
QUALIFIED
SITE_SURVEY
QUOTATION
NEGOTIATION
WON
LOST
    
```

15. Site Survey Module

This would make the project particularly useful for engineering companies.

Before preparing a quotation, engineers may conduct a site survey.

Example:

```

Survey ID:
SUR-2026-0014

Customer:
ABC University

Location:
    
```

Lahore Campus

Engineer :
Ali Raza

Survey Date :
03-Sep-2026

Survey can contain:

- photos,
- measurements,
- site conditions,
- existing equipment,
- electrical requirements,
- networking requirements,
- safety risks,
- suggested equipment,
- required labour,
- notes.

After completing the survey:

```
Site Survey
  ↓
Technical Assessment
  ↓
Bill of Materials
  ↓
Quotation
```

16. Quotation Management

Create customer quotations with:

- Products
- Services
- Quantity
- Unit price
- Discount
- Tax
- Installation charge
- Terms and conditions
- Validity period

Statuses:

DRAFT
SENT
VIEWED
ACCEPTED
REJECTED
EXPIRED

Accepted quotation can automatically become:

```
Quotation
  ↓
Sales Order
  ↓
Contract
  ↓
Project
```

17. Contract Management

Contracts should contain:

- Customer
- Contract number
- Start date
- End date

- Contract value
- Payment terms
- Project
- SLA
- Warranty conditions
- Maintenance agreement
- Attachments

Contract statuses:

```
DRAFT
UNDER_REVIEW
ACTIVE
EXPIRING
EXPIRED
TERMINATED
```

The system should generate reminders before expiry.

18. Project Management

Projects should be one of the largest modules.

Project record:

```
Project ID:
PRJ-2026-0018

Customer:
ABC University

Project:
Campus CCTV Upgrade

Project Manager:
Ahmed Khan

Contract Value:
PKR 5,000,000

Start:
10-Sep-2026

Deadline:
15-Nov-2026

Status:
IN_PROGRESS
```

Project functionality:

- Project phases
- Tasks
- Milestones
- Project team
- Sites
- Budgets
- Materials
- Purchase requests
- Expenses
- Assets
- Documents
- Risks
- Issues
- Progress
- Invoices
- Payments

19. Project Task Management

Projects contain tasks.

Example:

```
Project
├── Site Survey
├── Material Procurement
├── Cable Installation
├── Camera Installation
├── Network Configuration
├── Testing
└── Customer Handover
```

Each task can contain:

- Assignee
- Start date
- Deadline
- Priority
- Dependencies
- Completion percentage
- Notes
- Attachments

Statuses:

```
NOT_STARTED
IN_PROGRESS
BLOCKED
COMPLETED
CANCELLED
```

20. Project Milestones

Example:

```
Milestone 1
Material delivered

Milestone 2
50 cameras installed

Milestone 3
All buildings completed

Milestone 4
Customer acceptance
```

This allows management to track project progress without checking every individual task.

21. Bill of Materials

Every project can have a Bill of Materials.

Example:

ProductRequired	
IP Camera	100
NVR	10
CAT6 Cable	35 boxes
Network Switch	12
Rack	5
UPS	10

The ERP compares required quantity against available inventory.

```
Required: 100 Cameras
Available: 65
```

Shortage:
35 Cameras

The shortage can create a material request.

22. Procurement Management

Procurement should implement a professional flow.

```

Material Requirement
  ↓
Purchase Request
  ↓
Approval
  ↓
Request for Quotation
  ↓
Supplier Quotations
  ↓
Quotation Comparison
  ↓
Supplier Selection
  ↓
Purchase Order
  ↓
Goods Received Note
  ↓
Supplier Invoice
  ↓
Payment

```

This extends the realistic procurement approach established in the initial project design.

23. Purchase Request

Employees create purchase requests.

Example:

```

PR-2026-0031

Department:
Projects

Requested By:
Project Manager

Project:
PRJ-2026-0018

Required Date:
20-Sep-2026

Items:
35 IP Cameras
5 Network Switches

```

Possible statuses:

```

DRAFT
SUBMITTED
UNDER_REVIEW
APPROVED
REJECTED
CONVERTED_TO_RFQ

```

24. RFQ Management

Approved purchase requests can become Requests for Quotation.

The procurement officer selects multiple vendors.

Example:

RFQ-2026-0081

Vendors Invited:
Vendor A
Vendor B
Vendor C

Suppliers submit:

- Unit price
- Delivery time
- Payment terms
- Warranty
- Validity

25. Supplier Quotation Comparison

Instead of manually comparing Excel files, NEXORA can display:

Supplier	Cost	Delivery	Warranty
Vendor A	850,000	7 days	2 years
Vendor B	810,000	15 days	1 year
Vendor C	875,000	4 days	3 years

The procurement officer selects the preferred supplier.

No AI is necessary.

Selection remains a human business decision.

26. Vendor Management

Each supplier profile contains:

- Supplier code
- Company name
- Contacts
- Products
- Purchase history
- RFQs
- Quotations
- Purchase orders
- Deliveries
- Invoices
- Payments
- Return history

Vendor performance can be calculated through predefined rules.

Example:

On-Time Delivery: 92%
Rejected Products: 2%
Average Delivery: 6 Days
Orders Completed: 48

27. Purchase Orders

Approved supplier quotation becomes a purchase order.

PO statuses:

DRAFT
APPROVAL_PENDING
APPROVED
SENT
PARTIALLY_RECEIVED
RECEIVED
CLOSED
CANCELLED

28. Goods Receiving

When equipment arrives:

```
Purchase Order
  ↓
Goods Received Note
  ↓
Quality Inspection
  ↓
Warehouse Stock
```

The GRN records:

- PO number
- Supplier
- Receiving warehouse
- Product
- Ordered quantity
- Received quantity
- Damaged quantity
- Accepted quantity
- Serial numbers

29. Inventory Management

Inventory must be transaction-based rather than simple Add/Edit/Delete product forms, which was a key requirement of the initial design.

Products:

```
Product
Category
Brand
Model
SKU
Barcode
Serial Tracking
Unit
Cost
Sales Price
Minimum Stock
Maximum Stock
```

30. Multiple Warehouses

Example:

```
Central Warehouse
|
├── Lahore Warehouse
├── Karachi Warehouse
└── Islamabad Warehouse
```

Warehouse areas can also be structured:

```
Warehouse
  ↓
Zone
  ↓
Rack
  ↓
Shelf
  ↓
Bin
```

31. Stock Transactions

Transaction types:

```
PURCHASE_RECEIPT
STOCK_TRANSFER
```

```
PROJECT_ISSUE
PROJECT_RETURN
TECHNICIAN_ISSUE
TECHNICIAN_RETURN
SALES_ISSUE
DAMAGED
ADJUSTMENT
CUSTOMER_INSTALLATION
```

Every stock movement should create an immutable ledger entry.

32. Serial Number Tracking

Certain equipment should be individually tracked.

Example:

```
Product:
Hikvision Camera

SKU:
CAM-HIK-004

Serial:
SN98218271
```

Lifecycle:

```
Supplier
↓
Warehouse
↓
Project
↓
Technician
↓
Customer Site
↓
Installed Asset
```

33. Batch and Lot Tracking

For consumable products:

```
CAT6 Cable
Batch:
CB-2026-014

Received:
100 Boxes

Supplier:
ABC Cable

Manufactured:
2026

Remaining:
37
```

34. Stock Reservation

Projects should be able to reserve stock.

Example:

```
Available Cameras: 100

Project A Reserved: 40
Project B Reserved: 30

Free Stock:
30
```

This prevents another project from accidentally consuming reserved equipment.

35. Reorder Levels

Each product has:

Minimum Stock: 20

Current Stock: 12

System generates:

LOW STOCK ALERT

It can create a suggested replenishment request.

Again, this is deterministic business logic.

36. Asset Management

Asset management remains one of the key differentiators.

An installed physical product becomes an asset.

The original design already emphasized lifecycle traceability from purchase through installation, maintenance, replacement, and retirement.

Asset record:

Asset ID

Asset Type

Product

Serial Number

Customer

Site

Building

Floor

Room

Project

Installation Date

Warranty

Current Status

Assigned Technician

Purchase Cost

Supplier

37. Asset Lifecycle

PROCURED

↓

IN_WAREHOUSE

↓

ALLOCATED

↓

ISSUED

↓

INSTALLED

↓

ACTIVE

↓

UNDER_MAINTENANCE

↓

REPAIRED

↓

REPLACED

↓

RETIRED

38. QR Asset Tracking

Every asset should receive a unique QR code.

The original system concept uses the QR code to expose asset information and service history to authorized technicians.

Scanning could display:

```
Asset:
AST-000145

Customer:
ABC University

Site:
Building A

Location:
Floor 2 - Corridor

Model:
Hikvision DS-2CD2143

Installed:
18-Aug-2026

Warranty:
ACTIVE

Last Service:
02-Jun-2027

Next Service:
02-Dec-2027
```

Actions:

```
View Asset
Report Problem
Create Service Visit
Record Maintenance
Replace Asset
View History
```

39. Customer Site Management

Customers may have many sites.

Example:

```
ABC Bank

├─ Head Office
├─ Lahore Branch
├─ Islamabad Branch
├─ Karachi Branch
└─ Multan Branch
```

Each site can contain:

```
Site
↓
Building
↓
Floor
↓
Room / Area
↓
Assets
```

This creates precise physical asset mapping.

40. Preventive Maintenance

Maintenance can be scheduled according to:

- fixed date,
- monthly,
- quarterly,
- six-monthly,
- yearly,

- usage cycle,
- contract requirement.

Example:

```
Asset:
Generator G-001

Maintenance Cycle:
Every 90 Days

Last Service:
01-Aug-2026

Next Service:
30-Oct-2026
```

NEXORA generates upcoming-maintenance tasks.

41. Corrective Maintenance

Corrective maintenance occurs when something fails.

```
Customer Ticket
↓
Issue Diagnosis
↓
Work Order
↓
Technician Assignment
↓
Site Visit
↓
Repair
↓
Testing
↓
Customer Confirmation
↓
Closure
```

42. Helpdesk and Ticketing

Customers and employees can create tickets.

Categories:

```
Technical Issue
Installation Issue
Maintenance Request
Warranty Claim
Billing Issue
Network Problem
Equipment Failure
General Request
```

Priorities:

```
LOW
MEDIUM
HIGH
CRITICAL
```

Statuses:

```
OPEN
ASSIGNED
IN_PROGRESS
WAITING_CUSTOMER
WAITING_VENDOR
RESOLVED
CLOSED
```

43. SLA Management

This would make the ERP much more enterprise-oriented.

Example:

```
Priority: CRITICAL

Response SLA:
30 minutes

Resolution SLA:
4 hours
```

The system tracks:

```
SLA Remaining
SLA Breached
Response Time
Resolution Time
```

Management can generate SLA compliance reports.

44. Work Order Management

Work orders connect tickets with technicians.

The initial concept included a full work-order lifecycle from creation through assignment, dispatch, onsite work, resolution, customer confirmation, and closure.

Extended workflow:

```
NEW
↓
VALIDATED
↓
ASSIGNED
↓
TECHNICIAN_ACCEPTED
↓
TRAVELLING
↓
ON_SITE
↓
DIAGNOSIS
↓
WORK_IN_PROGRESS
↓
WAITING_FOR_PART
↓
RESOLVED
↓
CUSTOMER_CONFIRMATION
↓
CLOSED
```

45. Technician Management

Each technician profile can show:

- Skills
- Certifications
- Current project
- Assigned tickets
- Available status
- Branch
- Service history
- Performance

Availability:

```
AVAILABLE
ASSIGNED
ON_SITE
```

ON_LEAVE
OFF_DUTY

46. Technician Spare-Part Inventory

Technicians can carry inventory.

Example:

```
Technician:
Ahmed

Carried Stock:

RJ45 Connector      30
Network Cable       2
Power Adapter       3
Camera              1
```

When Ahmed uses one camera:

```
Technician Stock
      ↓
Work Order
      ↓
Customer Asset
```

The transaction automatically updates inventory.

47. Field Service Report

After completing work, technician submits:

```
Work Performed
Parts Used
Arrival Time
Departure Time
Root Cause
Resolution
Before Photo
After Photo
Customer Signature
Technician Signature
```

PDF service report can be generated.

48. Warranty Management

Assets should track warranties.

Statuses:

ACTIVE
EXPIRING
EXPIRED
VOID

The system can alert:

Warranty for AST-00182 expires in 30 days.

Warranty claims can be recorded against suppliers.

49. RMA / Equipment Return Management

Another strong commercial feature.

If defective equipment must return to supplier:

```
Faulty Asset
      ↓
RMA Request
      ↓
Supplier Approval
      ↓
Equipment Returned
```

```

↓
Repair / Replacement
↓
Equipment Received
↓
Asset Updated
    
```

50. Financial Management

Finance should connect directly to operational modules.

Core areas:

- Customer invoices
- Supplier invoices
- Payments
- Expenses
- Taxes
- Accounts
- Financial periods
- Credit notes
- Debit notes
- Project costing
- Budget tracking

51. Customer Invoice

Invoices may originate from:

```

Sales Order
Project
Contract
Maintenance
Work Order
    
```

Invoice status:

```

DRAFT
APPROVED
SENT
PARTIALLY_PAID
PAID
OVERDUE
CANCELLED
    
```

52. Supplier Invoice

Supplier invoice can be matched against:

```

Purchase Order
+
Goods Received Note
+
Supplier Invoice
    
```

This creates a three-way matching control.

If values do not match, finance can flag the invoice.

53. Expense Management

Employees can submit expenses.

Examples:

- Fuel
- Transport
- Accommodation
- Meals
- Tools
- Project materials
- Office purchases

Workflow:

```

Employee Expense
  ↓
Manager Approval
  ↓
Finance Verification
  ↓
Payment

```

Expenses can be assigned to a project.

54. Project Costing

This remains one of NEXORA's strongest capabilities.

The original concept calculates project profitability from material, labour, transport, subcontracting, and other costs.

Extended cost calculation:

```

Project Revenue

- Material Cost
- Employee Cost
- Technician Labour
- Transportation
- Fuel
- Accommodation
- Subcontractor
- Equipment Rental
- Purchase Expenses
- Miscellaneous Expenses
-----
Project Profit

```

Dashboard:

```

Contract Value:   PKR 5,000,000
Total Cost:       PKR 3,100,000
Gross Profit:     PKR 1,900,000
Profit Margin:    38%
Budget Used:      72%
Project Completion: 68%

```

55. Budget Management

Project managers can define:

```

Material Budget
Labour Budget
Transport Budget
Subcontractor Budget
Miscellaneous Budget

```

NEXORA compares:

```

Budget
vs
Actual
vs
Committed Cost

```

Example:

```

Material Budget:
2,000,000

Spent:
1,200,000

Purchase Orders Pending:
500,000

Available:
300,000

```

56. Approval Engine

The approval engine should be configurable rather than hard-coded.

The original concept already defined amount-based approval routes.

Example:

```
Purchase < 25,000
Manager Approval

25,000 - 250,000
Manager + Finance

250,000 - 1,000,000
Manager + Finance + General Manager

> 1,000,000
Manager + Finance + Director
```

But you can extend it to other operations:

- Purchase requests
- Purchase orders
- Expenses
- Quotations
- Discounts
- Leave requests
- Invoices
- Inventory adjustments
- Asset disposal
- Vendor onboarding

57. Workflow Engine

An even more advanced extension is a generic workflow engine.

Administrator can configure:

```
Object:
Purchase Request

Trigger:
Amount > 500,000

Step 1:
Procurement Manager

Step 2:
Finance Manager

Step 3:
Director

On Approval:
Create RFQ

On Rejection:
Return to Requester
```

This makes NEXORA configurable without changing application code.

58. Document Management

Documents can be attached to:

- Employees
- Customers
- Vendors
- Projects
- Contracts

- Purchase orders
- Invoices
- Assets
- Work orders

Document metadata:

```
Document Name
Category
Owner
Upload Date
Expiry Date
Version
Access Level
```

Examples:

- Contracts
- Invoices
- IDs
- Certifications
- Warranties
- Delivery notes
- Project drawings
- Inspection reports

59. Document Expiry Management

NEXORA can monitor:

```
Employee Certification Expiry
Vendor License Expiry
Contract Expiry
Warranty Expiry
Insurance Expiry
Equipment Certification Expiry
```

Alert:

```
Electrical Safety Certificate
expires in 15 days.
```

60. Customer Portal

The original system includes a restricted customer portal containing projects, sites, assets, invoices, contracts, service requests, warranties, and documents.

Expanded portal:

```
Dashboard
My Projects
My Contracts
My Sites
My Assets
My Tickets
My Work Orders
My Maintenance Schedule
My Invoices
My Payments
My Warranties
My Documents
```

Customer can:

- submit tickets,
- approve completed work,
- download invoices,
- download reports,
- view service history,
- view project progress.

61. Vendor Portal

The initial design similarly provided suppliers access to RFQs, purchase orders, deliveries, invoices, and payments.

Expanded functionality:

```
Vendor Dashboard
New RFQs
Submitted Quotations
Purchase Orders
Delivery Schedule
Goods Received
Rejected Items
Invoices
Payments
Performance
Documents
```

62. Notification Centre

System notifications can include:

```
Purchase Request Awaiting Approval
Low Inventory
Project Deadline Approaching
Invoice Overdue
Contract Expiring
Warranty Expiring
Ticket Assigned
Work Order Assigned
Maintenance Due
Leave Request Pending
RFQ Closing Soon
```

Channels:

- In-app
- Email
- Optional SMS

No AI required.

63. Audit Trail

Every sensitive operation should create an audit event, consistent with the initial system requirement.

Example:

```
User:
Ahmed Khan

Action:
PURCHASE_ORDER_APPROVED

Object:
PO-2026-00184

Previous State:
PENDING_APPROVAL

New State:
APPROVED

IP:
10.10.12.17

Device:
Chrome / Windows

Timestamp:
28-Aug-2026 17:40:18
```

Audit events should exist for:

- Logins
- Failed logins
- Approvals
- Rejections
- Invoice changes
- Payment changes
- Inventory adjustments
- Asset deletions
- Permission changes
- User creation
- Password changes

64. Maker-Checker Control

For financial and sensitive operations:

```
User A
Creates Transaction

↓

User B
Approves Transaction
```

The creator cannot approve their own high-risk transaction.

Useful for:

- Vendor creation
- Payments
- Purchase orders
- Inventory adjustments
- Asset disposal

This adds enterprise-grade internal control.

65. Fraud and Control Rules

Without AI, rule-based controls can detect suspicious operations.

Example:

```
If:
Invoice Amount > PO Amount

Then:
BLOCK APPROVAL
```

Another:

```
If:
User Creates Vendor
AND
Same User Creates Payment

Then:
Require Secondary Approval
```

Another:

```
If:
Stock Adjustment > 50 Units

Then:
Warehouse Manager Approval Required
```

These are business controls rather than AI.

66. Dashboard System

Different roles should receive different dashboards.

The original concept already separated CEO, Project Manager, Warehouse, and Maintenance dashboards.

CEO Dashboard

Revenue
Expenses
Net Profit
Outstanding Receivables
Outstanding Payables
Active Projects
Delayed Projects
Project Profitability
Inventory Value
Customer Count
Employee Count
Open Tickets

Finance Dashboard

Invoices
Payments
Expenses
Receivables
Payables
Overdue Accounts
Cash Position
Monthly Revenue
Monthly Expenses

Project Dashboard

Project Progress
Budget Utilization
Material Consumption
Milestones
Team Workload
Project Expenses
Project Profitability

Procurement Dashboard

Pending PRs
Pending Approvals
Open RFQs
Purchase Orders
Supplier Performance
Pending Deliveries

Warehouse Dashboard

Stock Value
Low Stock
Zero Stock
Inbound Stock
Outbound Stock
Reserved Stock
Pending Transfers
Damaged Stock

Maintenance Dashboard

Open Work Orders
Critical Tickets
SLA Violations
Upcoming Maintenance
Technician Availability
Expired Warranties

67. Reporting System

Reports should be exportable to:

- PDF

- Excel
- CSV

Reports can include:

HR

- Attendance report
- Leave report
- Employee report
- Payroll report

Procurement

- Purchase report
- Vendor report
- RFQ report
- PO report

Inventory

- Stock ledger
- Inventory valuation
- Product movement
- Low-stock report
- Damaged stock

Project

- Project status
- Budget vs actual
- Material usage
- Project profitability

Maintenance

- Asset maintenance
- Technician productivity
- SLA compliance
- Failure history

Finance

- Revenue
- Expenses
- Receivables
- Payables
- Profitability

68. Global Search

A professional ERP should include global search.

User types:

AST-00145

Results:

Asset AST-00145
Serial SN982182
Customer ABC University
Project PRJ-0018
Work Order W0-0048

Search should support:

- Employee IDs
- Customer names
- Project IDs
- Serial numbers

- Asset IDs
- Invoice IDs
- Purchase orders
- Tickets

69. Activity Timeline

Every major entity should contain a timeline.

Example project timeline:

```
10 Sep
Project Created

12 Sep
Purchase Request Submitted

14 Sep
Purchase Approved

18 Sep
Material Received

20 Sep
Installation Started

28 Sep
Milestone Completed

03 Oct
Customer Inspection
```

This gives users one chronological view.

70. Comments and Internal Notes

Users should be able to discuss records internally.

Example:

```
Project PRJ-0018

Ahmed:
Waiting for switches from vendor.

Ali:
Vendor confirmed delivery tomorrow.

Manager:
Installation team should continue Building B.
```

Mentions can notify employees.

71. Internal Task Management

Employees can create tasks independent of projects.

Example:

```
Task:
Verify supplier quotation

Assigned To:
Procurement Officer

Deadline:
Tomorrow

Priority:
High
```

72. Calendar

Unified calendar can display:

- Project deadlines
- Maintenance dates
- Employee leave
- Site visits
- Work orders
- Contract expirations
- Meetings
- Payment deadlines

73. Business Rules Engine

Another advanced feature:

Administrators define rules.

```
IF inventory < minimum_stock
THEN create low_stock_alert

IF invoice_due_date < today
AND invoice_status != PAID
THEN mark OVERDUE

IF warranty_expiry <= 30 days
THEN notify account_manager
```

This provides automation without artificial intelligence.

74. Scheduled Jobs

Background scheduled operations may perform:

```
Every Night:
Update overdue invoices

Every Morning:
Check expiring contracts

Every Hour:
Check SLA deadlines

Every Day:
Check maintenance schedules

Every Month:
Generate recurring invoices
```

This is system automation—not AI.

75. Recurring Contracts and Billing

Service companies often have recurring contracts.

Example:

```
Annual Maintenance Contract

Customer:
ABC Bank

Annual Value:
600,000

Billing:
Monthly

Monthly Invoice:
50,000
```

ERP automatically generates invoices according to a schedule.

76. Subscription / Service Package Management

Customers may purchase packages.

Example:

```
Gold Maintenance Plan
24/7 Support
4-hour response
Quarterly inspection
Free basic spare parts
```

Service entitlement is then validated against the customer's contract.

77. Asset Depreciation

For company-owned equipment:

```
Original Value:
PKR 500,000

Useful Life:
5 Years

Method:
Straight Line
```

The ERP calculates book value.

This could be optional depending on project scope.

78. Fixed Asset Management

Separate internal company assets from customer-installed assets.

Internal assets:

- Laptops
- Vehicles
- Tools
- Servers
- Office equipment
- Machinery

Assignment:

```
Laptop LT-0038
↓
Assigned To
↓
Employee EMP-0018
```

79. Fleet Management

This can extend NEXORA further for field companies.

Vehicles:

```
Vehicle
Registration
Driver
Mileage
Fuel
Maintenance
Insurance
Inspection
```

Vehicles can be assigned to technician teams.

Reports:

- Fuel consumption
- Vehicle maintenance
- Mileage

- Cost per vehicle

80. Tool Management

Field-service organizations also manage tools.

Example:

```
Fluke Cable Tester
Tool ID: TOOL-0018

Assigned:
Ahmed

Issue Date:
28-Aug

Expected Return:
31-Aug
```

This prevents expensive company tools from disappearing.

81. Safety and Incident Management

For industrial or field projects:

Employees can record:

- Safety incidents
- Near misses
- Site hazards
- Corrective actions
- PPE requirements

Example workflow:

```
Incident
↓
Supervisor Review
↓
Investigation
↓
Corrective Action
↓
Closure
```

82. Quality Inspection

Before customer handover:

```
Installation
↓
Internal Inspection
↓
Quality Checklist
↓
Pass / Fail
↓
Customer Acceptance
```

Inspection checklist example:

```
Camera mounted correctly ✓
Power stable ✓
Network connection ✓
Recording verified ✓
Cable labelled ✓
Area cleaned ✓
```

83. Customer Handover

Project completion should generate a handover package.

It can include:

- Installed asset list
- Serial numbers
- Site photographs
- Inspection report
- Warranty information
- Service contact
- Customer signature
- Completion certificate

84. Knowledge Base

Without AI, NEXORA can still contain a searchable knowledge base.

Categories:

```
Installation Guides
Troubleshooting Procedures
Safety Procedures
Product Manuals
Company Policies
SOPs
```

Technicians search manually.

85. Custom Fields

Different companies need different information.

Administrator can create:

```
Custom Field:
Camera Lens Type

Type:
Dropdown

Options:
2.8mm
4mm
6mm
```

Custom fields can be attached to:

- Customers
- Assets
- Projects
- Tickets
- Employees
- Products

This greatly improves product flexibility.

86. Custom Statuses

Organizations could define their own project or ticket statuses.

Example:

```
READY_FOR_SURVEY
CUSTOMER_APPROVAL_PENDING
WAITING_FOR_MATERIAL
INSTALLATION_READY
```

This avoids hard-coding every company workflow.

87. Localization

To make the ERP commercially useful:

Support:

- Multiple currencies
- Multiple time zones

- Multiple languages
- Local date formats
- Tax configurations

Possible languages:

```
English
Urdu
Arabic
```

88. Data Import and Export

Companies should be able to import:

- Employees
- Products
- Customers
- Vendors
- Inventory

Using:

```
CSV
Excel
```

Export should also be available.

89. Backup and Restore

Administrative features:

- Automated database backup
- Manual backup
- Backup history
- Restore point
- Backup status

90. System Health Dashboard

Super administrators can monitor:

```
Application Status
Database Status
Storage Usage
Failed Jobs
Email Queue
Backup Status
Active Users
API Errors
```

91. API Integration Layer

Even without AI, the system can expose APIs.

Example:

```
/api/customers
/api/projects
/api/assets
/api/work-orders
/api/invoices
```

This allows integration with:

- Mobile applications
- Accounting systems
- Payment gateways
- Biometric attendance
- SMS gateways
- external customer portals

92. Webhook System

NEXORA can generate webhooks.

Example:

```
Event:
invoice.paid

POST:
https://external-system.example/webhook
```

Possible events:

```
project.created
invoice.created
invoice.paid
ticket.created
asset.installed
purchase_order.approved
```

93. Mobile / PWA Application

Technicians should not need a laptop onsite.

PWA functions:

```
My Jobs
My Work Orders
Scan Asset QR
Customer Details
Navigation
Add Photos
Use Spare Part
Update Status
Customer Signature
Service Report
```

Offline functionality could be added later.

94. Offline Field Mode

Advanced extension:

Technician visits a location without internet.

The PWA stores:

```
Work Order
Asset Information
Checklist
Notes
Photos
Customer Signature
```

When internet returns:

```
Offline Data
↓
Synchronization
↓
ERP Database
```

95. Security Features

NEXORA should implement:

- HTTPS
- Password hashing
- MFA
- RBAC
- Rate limiting
- CSRF protection
- Secure cookies

- Input validation
- SQL injection protection
- XSS protection
- Audit logging
- Session expiration
- Account lockout
- Encryption of sensitive data
- Secure file uploads
- Backup protection

96. Data Isolation

For multi-tenant architecture:

```
Tenant A  
cannot access  
Tenant B
```

Every business record contains a tenant identifier.

This must be enforced at the backend—not simply hidden in the frontend.

101. Recommended Development Phases

Do not implement every module simultaneously.

Phase 1 — Core Platform

Build:

```
Authentication  
Organization  
Branches  
Departments  
Users  
Roles  
Permissions  
Audit Logs  
System Settings
```

Goal:

Create the secure ERP foundation.

Phase 2 — Business Directory

Build:

```
Employees  
Customers  
Customer Sites  
Vendors  
Products
```

Phase 3 — Inventory

Build:

```
Warehouses  
Stock  
Stock Ledger  
Serial Numbers  
Stock Transfer  
Stock Issue  
Stock Return  
Stock Adjustment
```

Phase 4 — Procurement

Build:

```
Purchase Request  
Approval
```

- RFQ
- Supplier Quotation
- Quotation Comparison
- Purchase Order
- GRN

Phase 5 — Projects

Build:

- Projects
- Tasks
- Milestones
- Teams
- Sites
- Budget
- Bill of Materials
- Material Allocation
- Project Expenses

Phase 6 — Asset Lifecycle

Build:

- Asset Registration
- Serial Tracking
- QR Codes
- Installation
- Asset History
- Warranty
- Replacement
- Retirement

Phase 7 — Field Operations

Build:

- Tickets
- SLA
- Work Orders
- Technicians
- Site Visits
- Service Reports
- Spare Parts

Phase 8 — Maintenance

Build:

- Preventive Maintenance
- Corrective Maintenance
- Maintenance Calendar
- Warranty Claims
- RMA

Phase 9 — Finance

Build:

- Customer Invoices
- Supplier Invoices
- Payments
- Expenses
- Project Costing
- Budget vs Actual
- Receivables
- Payables

Phase 10 — Enterprise Controls

Build:

- Approval Engine
- Workflow Engine

Maker-Checker
Audit Reports
Notification Centre
Document Management

Phase 11 — External Portals

Build:

Customer Portal
Vendor Portal
Technician PWA

Phase 12 — Advanced Features

Build:

Fleet
Tools
Safety
Quality Inspection
Contract Billing
Global Search
Custom Fields
Custom Workflows
API
Webhooks
Offline Mode
Localization

102. Final Navigation Structure

A professional sidebar could look like:

Dashboard

CRM

- └ Leads
- └ Opportunities
- └ Customers
- └ Quotations
- └ Contracts

Projects

- └ Projects
- └ Tasks
- └ Milestones
- └ Sites
- └ Budgets

Procurement

- └ Purchase Requests
- └ RFQs
- └ Quotations
- └ Purchase Orders
- └ GRNs

Inventory

- └ Products
- └ Warehouses
- └ Stock
- └ Transfers
- └ Stock Ledger

Assets

- └ Assets
- └ Installations
- └ Warranties
- └ Asset History

Service

- └ Tickets

- |— Work Orders
- |— Maintenance
- |— Technicians
- |— Service Reports

Finance

- |— Invoices
- |— Payments
- |— Expenses
- |— Receivables
- |— Payables

HR

- |— Employees
- |— Attendance
- |— Leave
- |— Payroll

Vendors

- |— Vendors
- |— Vendor Performance

Reports

Documents

Approvals

Notifications

Audit Logs

Administration

- |— Organization
- |— Branches
- |— Departments
- |— Roles
- |— Permissions
- |— Workflows
- |— System Settings

103. What Makes NEXORA Different

The uniqueness should not be marketed as:

Business example

We built an ERP with HR, Finance and Inventory.

Instead:

Business example

****NEXORA ERP is an integrated enterprise operations platform that connects sales, procurement, project delivery, serialized inventory, customer-site asset management, field operations, maintenance, finance, approvals and lifecycle traceability inside one unified system.****

The strongest differentiating concept is:

```

CUSTOMER
  ↓
CONTRACT
  ↓
PROJECT
  ↓
PROCUREMENT
  ↓
INVENTORY
  ↓
INSTALLATION
  ↓
    
```

ASSET
↓
MAINTENANCE
↓
FINANCE

Everything is connected.

104. Professional Portfolio Description

A strong final project description would be:

Business example

****NEXORA ERP is a multi-tenant enterprise resource planning platform designed for field-service, engineering and asset-driven organizations. The system manages end-to-end business operations including CRM, customer contracts, procurement, inventory and serialized stock tracking, project management, asset installation, QR-based asset lifecycle management, preventive and corrective maintenance, helpdesk tickets, technician work orders, HR, financial transactions, project costing, configurable approval workflows, customer/vendor portals, audit trails, SLA management, document management and executive reporting.****

105. Recommended Core Scope

Even though the full specification contains many modules, the most important modules to implement deeply are:

1. Authentication + RBAC
2. Organization Management
3. Customer Management
4. Vendor Management
5. Product Management
6. Warehouse Management
7. Inventory Ledger
8. Procurement
9. Approval Engine
10. Project Management
11. Project Budgeting
12. Asset Management
13. QR Asset Tracking
14. Customer Sites
15. Tickets
16. Work Orders
17. Maintenance
18. Technician Management
19. Invoices
20. Expenses
21. Project Profitability
22. Notifications
23. Documents
24. Audit Logs
25. Dashboards
26. Customer Portal
27. Vendor Portal

These 27 areas alone would already produce a very substantial professional ERP.

106. Final Vision

The finished NEXORA platform should allow management to answer questions such as:

Which projects are profitable?

Where is this device installed?

Who purchased this equipment?

Which vendor supplied it?

Which warehouse received it?

Which technician installed it?

Is the asset under warranty?

How many times has it failed?

What spare parts were used?

What did maintenance cost?

What invoices remain unpaid?

Which customers have expiring contracts?

Which projects are over budget?

Which purchase requests are waiting for approval?

Which technicians are available?

Which equipment requires maintenance next week?

What happened to an asset from purchase until retirement?

If your ERP can answer those questions reliably from one interconnected database, you have moved far beyond a basic student ERP.

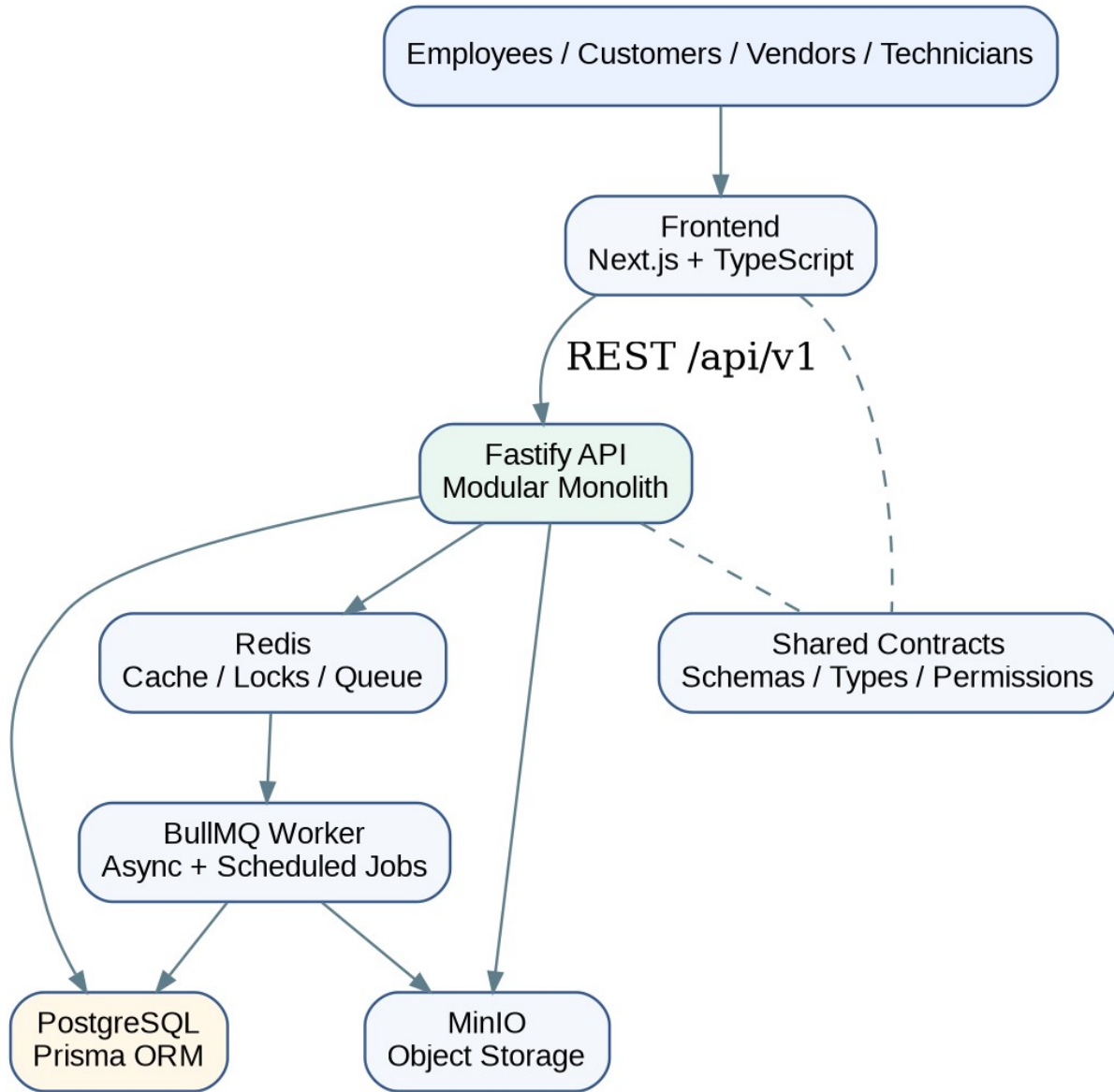
You are building an actual enterprise operations platform.

3. Final Technology Stack

Locked implementation stack

Frontend and backend remain in one TypeScript monorepo. The backend is Fastify + TypeScript. PostgreSQL is the system of record, Prisma is the ORM and migration layer, MiniIO is the private object store, Redis provides cache/locks/queue transport, and BullMQ handles asynchronous and scheduled work.

Layer	Technology	Primary responsibility
Frontend	Next.js + TypeScript	ERP web application, customer/vendor portals and PWA-capable technician UI
UI	Tailwind CSS + reusable component system	Consistent forms, tables, drawers, dialogs and dashboards
Server state	TanStack Query	API caching, mutations, invalidation, pagination and request lifecycle
Backend	Fastify + TypeScript	REST API, domain modules, authorization, transactions and business logic
Validation/contracts	Zod (shared)	Runtime validation plus inferred TypeScript request/response types
Database	PostgreSQL	Authoritative relational system of record
ORM	Prisma ORM	Type-safe persistence, migrations and transactions
Object storage	MiniIO	Private documents, images, reports, signatures and project files
Cache/locks/queue	Redis	Short-lived cache, distributed locks, rate-limit state and BullMQ transport
Background jobs	BullMQ	Email, PDFs, reminders, scheduled checks, exports, webhooks
Proxy	Nginx	TLS termination, routing, headers and request limits
Containers	Docker + Docker Compose	Repeatable local and initial production runtime
CI/CD	GitHub Actions	Lint, typecheck, tests, security checks, builds and deployment
IaC later	Terraform	Repeatable infrastructure provisioning



Final application architecture

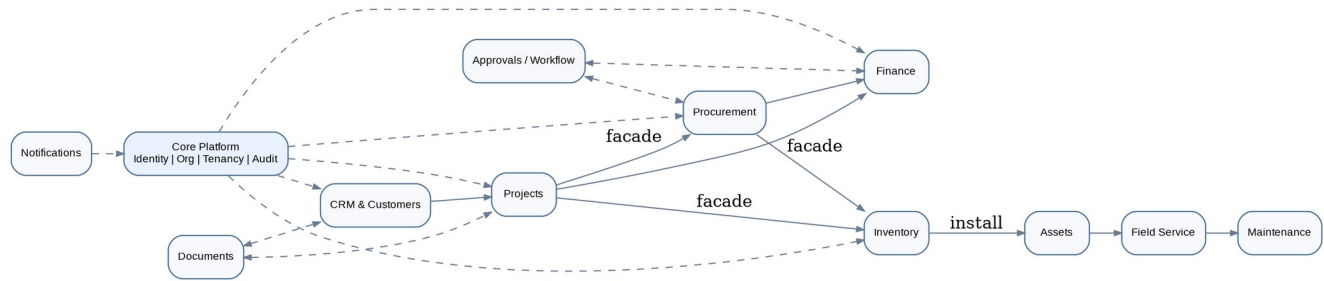
3.1 Dependency-management model

```

nexora-erp/
├── node_modules/      # root workspace installation
├── frontend/
├── backend/
├── worker/
├── shared/
├── database/
└── package.json
    
```

The requirement is one workspace-managed dependency environment at the repository root, not separately maintained frontend and backend dependency installations. Each workspace still declares the dependencies it owns in its package.json; the package manager resolves and links them from the workspace installation.

4. Monorepo and Opinionated Modular-Monolith Architecture



Major domain boundaries and approved cross-domain communication

```
nexora-erp/
├── frontend/
│   └── src/{app,modules,components,layouts,hooks,lib}
├── backend/
│   └── src/{config,core,plugins,modules,types}
├── worker/
│   └── src/{queues,processors,schedulers}
├── shared/
│   └── src/{contracts,schemas,enums,constants,permissions,utils}
├── database/
│   ├── prisma/{schema.prisma,models,migrations,seed}
│   └── src/client.ts
├── infrastructure/{docker,nginx,terraform}
├── docs/
├── tests/e2e/
└── .github/workflows/
```

4.1 Backend domain-first structure

```
backend/src/modules/procurement/
├── procurement.module.ts
├── procurement.facade.ts
├── procurement.permissions.ts
├── procurement.events.ts
├── index.ts
├── purchase-request/
│   ├── purchase-request.routes.ts
│   ├── purchase-request.controller.ts
│   ├── purchase-request.service.ts
│   ├── purchase-request.repository.ts
│   ├── purchase-request.mapper.ts
│   ├── purchase-request.errors.ts
│   └── purchase-request.test.ts
├── rfq/
├── quotation/
├── purchase-order/
└── goods-receipt/
```

Layer	Allowed responsibilities	Forbidden responsibilities
Route	URL, method, schema binding, pre-handlers, controller binding	Business rules, Prisma queries
Controller	Translate HTTP input/output, status codes	Business calculations, direct database access
Service	Business rules, orchestration, authorization scope checks, transaction coordination	HTTP response formatting
Repository	Persistence queries for module-owned entities	Calling controllers or exposing database types as API
Mapper	Persistence/domain to API DTO mapping	Database writes
Facade	Public synchronous API exposed to other modules	Leaking repositories/internal services
Event handler	Non-critical side effects and reactions	Replacing atomic business transactions

4.2 Mandatory module-boundary rules

- Every business feature belongs to exactly one owning domain module.
- Routes and controllers never call Prisma directly.
- Module A cannot import Module B repositories or private services.
- Synchronous cross-module calls use the target module public facade.
- Non-critical asynchronous side effects use domain events/BullMQ.
- Critical changes that must remain consistent are committed in one database transaction.
- Prisma-generated types are internal persistence types; shared API contracts are separate.
- Frontend imports only shared browser-safe packages, never backend or database packages.
- All tenant-owned queries require tenant context.
- Critical mutations emit a business audit event.

5. Cross-Module Rules and Transaction Boundaries

Workflow	Atomic transaction must include	Async after commit
Goods receipt	GRN + GRN items + serial/batch capture + stock transactions + PO received quantities + audit	Email, notification, export, webhook
Stock transfer	Transfer state + source decrement/ledger + destination increment/ledger + serial movement + audit	Notification
Purchase approval	Approval action + status transition + next approval state + audit	Approver notifications
Asset installation	Stock consumption/issue + serial state + asset record + installation history + audit	QR/PDF generation, customer notification
Invoice approval	Invoice final totals/status + approval + journal posting if enabled + audit	PDF generation, email, webhook
Payment posting	Payment + allocations + invoice balances/status + journal posting + audit	Receipt PDF/email
Work-order completion	Work order + service report + parts consumption + asset/service history + audit	Customer signature PDF, notification

Rule

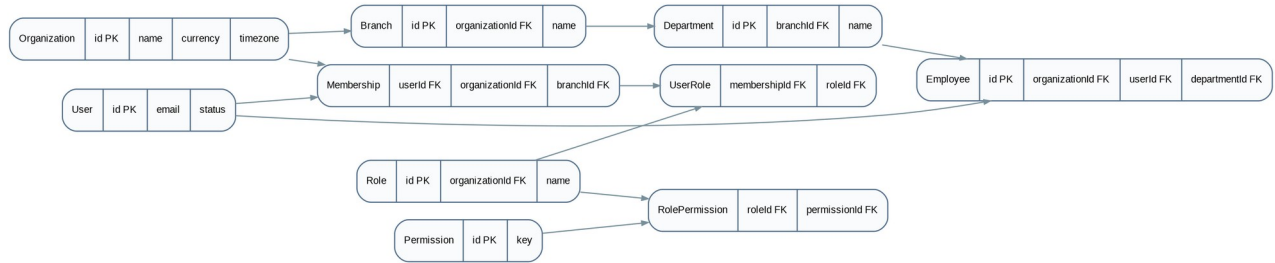
Never use eventual consistency for stock balances, invoice balances, approval state or accounting postings when the workflow can be committed inside the same PostgreSQL transaction.

6. Database Design Standards

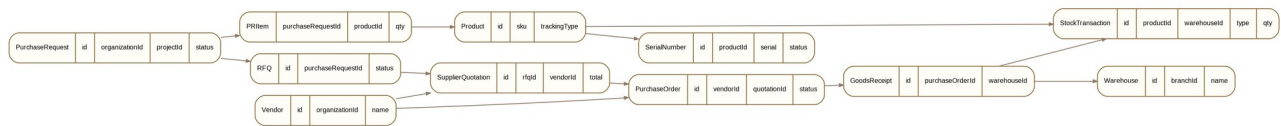
Standard	Decision
Primary keys	UUID identifiers generated consistently; time-sortable UUIDs are acceptable if standardized project-wide.
Tenant key	organizationId on every tenant-owned aggregate and transaction table.
Branch scope	branchId where records are branch-owned or operationally restricted.
Money	DECIMAL/NUMERIC, never floating point. Recommended 18,2 for currency and 18,4 where needed.
Quantities	NUMERIC to support fractional units; unit of measure stored explicitly.
Time	UTC timestamps in database; organization timezone used only for display/business calendars.
Soft delete	Allowed for master/reference data where retention is useful; not used to hide immutable ledgers.
Ledgers	Stock transactions, journal lines and audit logs are append-only or reversal-based.
Status transitions	Services enforce valid state-machine transitions; clients cannot freely PATCH status.
Concurrency	Use transactions, unique constraints and locking/version fields for inventory/approval races.
Indexes	Tenant key first in common compound indexes; index foreign keys, status/date queues and business identifiers.
Business numbers	Human-readable numbers (PR-..., PO-..., INV-...) are separate from primary keys and unique per organization.
References	Polymorphic subjectType/subjectId only where a true generic subsystem is required; otherwise explicit FKs preferred.

7. Database Entity Catalog and Relationship Mapping

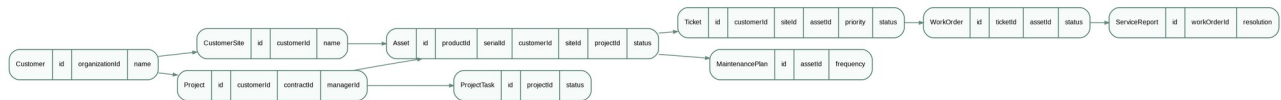
This catalog defines the target logical data model. It is intentionally broader than the minimum MVP so that module boundaries and future migrations are clear. Physical Prisma models may be introduced phase-by-phase while retaining these ownership rules.



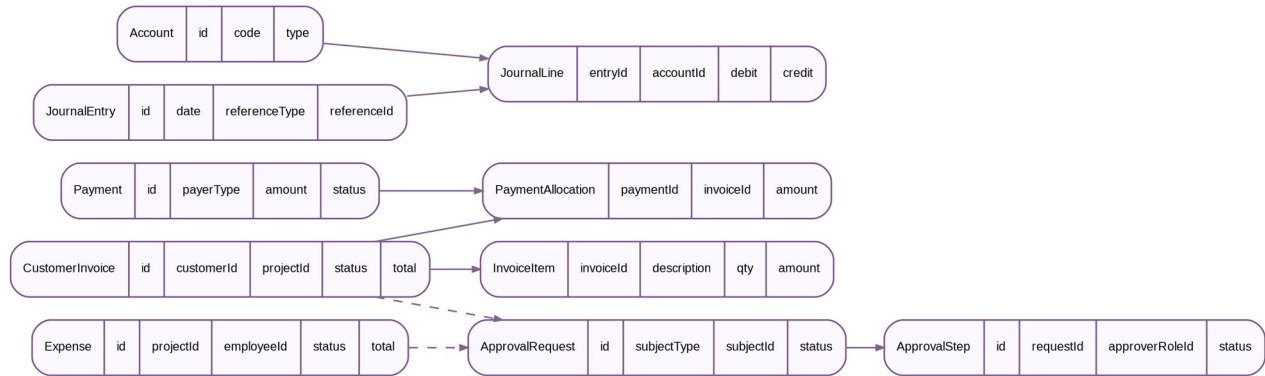
Core organization, identity and RBAC relationship map



Procurement and inventory relationship map



Project, asset and field-service relationship map



Finance and approval relationship map

Identity & Access

Entity	Key fields	Purpose	Major relationships
User	id, email, passwordHash, status, lastLoginAt	Global login identity	1:N Session; 1:N Membership
Session	id, userId, refreshTokenHash, device, ip, expiresAt, revokedAt	Revocable login session	N:1 User
MfaCredential	id, userId, type, secretEncrypted, enabledAt	MFA enrollment	N:1 User
Permission	id, key, description	Canonical permission key	M:N Role
Role	id, organizationId, name, systemRole	Tenant role	M:N Permission; M:N Membership
RolePermission	roleId, permissionId	Role-permission bridge	N:1 Role, Permission
OrganizationMembership	id, userId, organizationId, branchId, status	User participation in tenant	N:1 User/Organization
UserRole	membershipId, roleId	Role assignment in tenant	N:1 Membership/Role

Organization

Entity	Key fields	Purpose	Major relationships
Organization	id, code, name, currency, timezone, locale, status	Tenant root	1:N Branch/Customer/Vendor/Project/...
Branch	id, organizationId, code, name, addressId	Operating branch	N:1 Organization; 1:N Department/Warehouse
Department	id, organizationId, branchId, name, managerEmployeeId	Functional department	N:1 Branch; 1:N Employee
Team	id, organizationId, departmentId, name	Optional team grouping	N:1 Department; M:N Employee
Address	id, organizationId, type, lines, city, region, country	Reusable address	Linked by branch/customer/vendor/site
OrganizationSetting	id, organizationId, key, valueJson	Tenant configuration	N:1 Organization

HR

Entity	Key fields	Purpose	Major relationships
Employee	id, organizationId, userId, branchId, departmentId, employeeNo, managerId, status	Employee master	N:1 Org/Branch/Dept; self-manager
EmployeeSkill	employeeId, skillId, proficiency	Employee skill mapping	N:1 Employee/Skill
EmployeeCertification	id, employeeId, name, issuedAt, expiresAt, documentId	Certification history	N:1 Employee
Attendance	id, employeeId, workDate, checkIn, checkOut, status	Daily attendance	N:1 Employee
LeaveType	id, organizationId, name, annualAllowance	Leave policy type	1:N LeaveBalance/Request
LeaveBalance	id, employeeId, leaveTypeId, year, opening, used, remaining	Leave balance	N:1 Employee/LeaveType
LeaveRequest	id, employeeId, leaveTypeId, fromDate, toDate, status, approvalRequestId	Leave workflow	N:1 Employee; optional ApprovalRequest
PayrollRun	id, organizationId, periodStart, periodEnd, status	Payroll batch	1:N PayrollItem
PayrollItem	id, payrollRunId, employeeId, gross, deductions, net	Employee payroll result	N:1 PayrollRun/Employee

CRM & Customer

Entity	Key fields	Purpose	Major relationships
Lead	id, organizationId, ownerId, source, name, status	Prospect lead	May convert to Opportunity/Customer
Opportunity	id, organizationId, customerId, leadId, ownerId, stage, estimatedValue	Sales opportunity	N:1 Customer/Lead
SiteSurvey	id, organizationId, opportunityId, customerId, engineerId, status	Technical survey	N:1 Opportunity/Site
Quotation	id, organizationId, opportunityId, customerId, quoteNo, status, validUntil, totals	Commercial quotation	1:N QuotationItem
QuotationItem	id, quotationId, productId, description, qty, price, tax	Quotation line	N:1 Quotation
SalesOrder	id, organizationId, quotationId, customerId, orderNo, status, total	Accepted sale	May create Contract/Project
Contract	id, organizationId, customerId, salesOrderId, contractNo, startDate, endDate, value, status	Customer contract	1:N Project/Service entitlement
ServicePackage	id, organizationId, name, responseSla, resolutionSla, billingCycle	Service plan template	Referenced by Contract
Customer	id, organizationId, code, name, taxNo, status, creditLimit	Customer master	1:N Contacts/Sites/Projects/Invoices
CustomerContact	id, customerId, name, email, phone, isPrimary	Customer person	N:1 Customer
CustomerSite	id, organizationId, customerId, code, name, addressId	Service/project location	1:N Assets/Tickets
SiteBuilding	id, customerId, name	Building grouping	1:N SiteArea
SiteArea	id, buildingId, parentAreaId, name, areaType	Floor/room/zone hierarchy	1:N Asset

Vendors

Entity	Key fields	Purpose	Major relationships
Vendor	id, organizationId, code, name, taxNo, status, paymentTerms	Supplier master	1:N Quotes/POs/Invoices
VendorContact	id, vendorId, name, email, phone, isPrimary	Supplier person	N:1 Vendor
VendorProduct	vendorId, productId, vendorSku, leadTimeDays, lastPrice	Approved vendor-product link	N:1 Vendor/Product
VendorPerformance	id, vendorId, period, onTimePct, rejectPct, score	Periodic performance snapshot	N:1 Vendor

Product & Inventory

Entity	Key fields	Purpose	Major relationships
ProductCategory	id, organizationId, parentId, name	Product taxonomy	1:N Product
Product	id, organizationId, categoryId, sku, name, unitId, trackingType, minStock, maxStock	Item/service master	1:N stock/serial/batch lines
UnitOfMeasure	id, organizationId, code, name, precision	Quantity unit	1:N Product
Warehouse	id, organizationId, branchId, code, name, status	Stock location	1:N WarehouseLocation/StockBalance
WarehouseLocation	id, warehouseId, parentId, type, code, name	Zone/rack/shelf/bin tree	1:N balances/transactions
StockBalance	id, organizationId, warehouseId, locationId, productId, onHand, reserved	Materialized current balance	Unique scope per location/product
StockTransaction	id, organizationId, productId, warehouseId, locationId, type, qty, referenceType, referenceId, occurredAt	Immutable stock ledger	N:1 Product/Warehouse
StockReservation	id, organizationId, productId, warehouseId, projectId, qty, status	Reserved inventory	N:1 Product/Warehouse/Project
StockTransfer	id, organizationId, fromWarehouseId, toWarehouseId, transferNo, status	Warehouse movement header	1:N StockTransferItem
StockTransferItem	id, transferId, productId, qty, receivedQty	Transfer line	N:1 Transfer/Product
SerialNumber	id, organizationId, productId, serialNo, status, currentWarehouseId, assetId	Individually tracked unit	N:1 Product; optional Asset
BatchLot	id, organizationId, productId, lotNo, manufactureDate, expiryDate, qtyRemaining	Lot-tracked stock	N:1 Product
StockAdjustment	id, organizationId, warehouseId, reason, status, approvalRequestId	Controlled correction	1:N adjustment lines

Procurement

Entity	Key fields	Purpose	Major relationships
MaterialRequirement	id, organizationId, projectId, requestedById, status	Project/department material need	1:N requirement items
PurchaseRequest	id, organizationId, branchId, projectId, prNo, requesterId, requiredDate, status, approvalRequestId	Internal purchase request	1:N PRItem; may create RFQ
PurchaseRequestItem	id, purchaseRequestId, productId, description, qty, estimatedPrice	PR line	N:1 PR/Product
RFQ	id, organizationId, purchaseRequestId, rfqNo, closesAt, status	Supplier bidding request	M:N Vendor via RFQVendor
RFQVendor	rfqId, vendorId, invitedAt, responseStatus	RFQ invitation	N:1 RFQ/Vendor
SupplierQuotation	id, organizationId, rfqId, vendorId, quoteRef, validity, total, status	Vendor commercial response	1:N SupplierQuotationItem
SupplierQuotationItem	id, supplierQuotationId, productId, qty, unitPrice, deliveryDays, warrantyMonths	Vendor line quote	N:1 SupplierQuotation/Product
PurchaseOrder	id, organizationId, vendorId, supplierQuotationId, poNo, orderDate, expectedDate, status, total	Purchase commitment	1:N POItem/GRN
PurchaseOrderItem	id, purchaseOrderId, productId, orderedQty, receivedQty, unitPrice, tax	PO line	N:1 PO/Product
GoodsReceipt	id, organizationId, purchaseOrderId, warehouseId, grnNo, receivedAt, receivedById, status	Goods receiving header	1:N GRNItem
GoodsReceiptItem	id, goodsReceiptId, poItemid, productId, receivedQty, acceptedQty, damagedQty	Received line	Creates stock ledger
QualityInspection	id, organizationId, goodsReceiptId	Receiving quality check	N:1 GoodsReceipt

Entity	Key fields	Purpose	Major relationships
	inspectorId, result, notes		

Projects

Entity	Key fields	Purpose	Major relationships
Project	id, organizationId, customerId, contractId, siteld, projectNo, managerId, status, startDate, dueDate, contractValue	Project aggregate root	1:N tasks/members/budgets/assets
ProjectPhase	id, projectId, name, sequence, status	Project phase	N:1 Project
ProjectTask	id, projectId, phaseld, assigneeld, title, status, priority, startDate, dueDate, completionPct	Work item	N:1 Project/Phase
ProjectTaskDependency	taskId, dependsOnTaskId, type	Task dependency	self M:N
ProjectMilestone	id, projectId, name, dueDate, achievedAt, status	Milestone	N:1 Project
ProjectMember	projectId, employeeId, role, allocationPct	Project team	N:1 Project/Employee
BillOfMaterials	id, projectId, version, status	Project BOM header	1:N BOMItem
BOMItem	id, billOfMaterialsId, productId, requiredQty, reservedQty, issuedQty	Material plan line	N:1 BOM/Product
ProjectBudget	id, projectId, version, status, totalBudget	Budget header	1:N ProjectBudgetLine
ProjectBudgetLine	id, projectBudgetId, category, budgetAmount, committedAmount, actualAmount	Budget category	N:1 Budget
ProjectExpense	id, projectId, expenseId, costCategory	Project cost link	N:1 Project/Expense
ProjectRisk	id, projectId, title, probability, impact, ownerId, status	Risk register	N:1 Project
ProjectIssue	id, projectId, title, severity, ownerId, status	Issue register	N:1 Project
ProjectHandover	id, projectId, acceptedByCustomerId, acceptedAt, documentId, status	Project completion evidence	N:1 Project

Assets & Field Service

Entity	Key fields	Purpose	Major relationships
Asset	id, organizationId, assetNo, productId, serialNumberId, customerId, siteld, areald, projectId, status, installedAt	Installed/customer asset	1:N history/tickets/maintenance
AssetInstallation	id, assetId, projectId, technicianId, installedAt, locationText, checklistId	Installation evidence	N:1 Asset
AssetHistory	id, assetId, eventType, oldStatus, newStatus, referenceType, referenceId, occurredAt	Asset lifecycle log	N:1 Asset
AssetWarranty	id, assetId, vendorId, startsAt, expiresAt, terms, status	Warranty coverage	N:1 Asset/Vendor
AssetQrTag	id, assetId, token, generatedAt, revokedAt	QR lookup token	1:1 Asset
AssetRMA	id, assetId, vendorId, rmaNo, status, sentAt, returnedAt, resolution	Vendor return process	N:1 Asset
Ticket	id, organizationId, ticketNo, customerId, siteld, assetId, category, priority, status, openedById, slaPolicyId	Support request	1:N comments/work orders
TicketComment	id, ticketId, authorId, visibility, body, createdAt	Ticket conversation	N:1 Ticket
SlaPolicy	id, organizationId, name, priority, responseMinutes, resolutionMinutes	SLA rule	1:N Ticket
WorkOrder	id, organizationId, workOrderNo, ticketId, assetId, projectId, status, scheduledAt, priority	Field job	1:N assignments/reports
WorkOrderAssignment	id, workOrderId, technicianId, assignedAt, acceptedAt	Technician assignment	N:1 WorkOrder/Employee
TechnicianProfile	employeeId, availabilityStatus, homeBranchId	Technician extension	1:1 Employee
ServiceReport	id, workOrderId, technicianId, arrivalAt, departureAt, rootCause, resolution, customerSignDocumentId	Completion report	N:1 WorkOrder
ServiceReportPart	id, serviceReportId, productId, qty, stockTransactionId	Part used	N:1 report/product

Maintenance

Entity	Key fields	Purpose	Major relationships
MaintenancePlan	id, organizationId, assetId, contractId, frequencyType, intervalValue, active	Preventive plan	1:N schedules
MaintenanceSchedule	id, maintenancePlanId, dueAt, status, generatedWorkOrderId	Planned occurrence	N:1 plan
MaintenanceExecution	id, scheduleId, workOrderId, completedAt, result, nextDueAt	Executed maintenance	N:1 schedule
MaintenanceChecklist	id, organizationId, name, version	Checklist template	1:N checklist items
MaintenancePart	id, maintenanceExecutionId, productId, qty, stockTransactionId	Maintenance consumable	N:1 execution/product

Finance & Accounting

Entity	Key fields	Purpose	Major relationships
CustomerInvoice	id, organizationId, invoiceNo, customerId, projectId, contractId, issueDate, dueDate, status, subtotal, tax, total, balance	Accounts receivable invoice	1:N items/allocations
CustomerInvoiceItem	id, invoiceId, productId, description, qty, unitPrice, tax, lineTotal	AR invoice line	N:1 invoice
SupplierInvoice	id, organizationId, invoiceNo, vendorId, purchaseOrderId, goodsReceiptId, status, total, matchStatus	Accounts payable invoice	1:N items/payments
SupplierInvoiceItem	id, supplierInvoiceId, poltemId, description, qty, unitPrice, lineTotal	AP invoice line	N:1 supplier invoice
Payment	id, organizationId, paymentNo, direction, partyType, partyId, amount, method, paidAt, status	Cash settlement	1:N allocations
PaymentAllocation	id, paymentId, invoiceType, invoiceId, amount	Payment-to-invoice bridge	N:1 Payment/Invoice
Expense	id, organizationId, employeeId, projectId, category, incurredAt, status, total, approvalRequestId	Employee/operating expense	1:N ExpenseItem
ExpenseItem	id, expenseId, description, amount, tax, documentId	Expense detail	N:1 Expense
FinancialPeriod	id, organizationId, startDate, endDate, status	Accounting period	1:N JournalEntry
Account	id, organizationId, code, name, type, parentId, active	Chart of accounts	1:N JournalLine
JournalEntry	id, organizationId, periodId, entryNo, postedAt, referenceType, referenceId, status	Accounting posting header	1:N JournalLine
JournalLine	id, journalEntryId, accountId, debit, credit, projectId, branchId	Double-entry line	N:1 Entry/Account
CreditNote	id, organizationId, customerInvoiceId, noteNo, total, status	Customer credit adjustment	N:1 invoice
DebitNote	id, organizationId, supplierInvoiceId, noteNo, total, status	Supplier debit adjustment	N:1 supplier invoice

Approvals, Documents & Platform

Entity	Key fields	Purpose	Major relationships
ApprovalDefinition	id, organizationId, subjectType, name, conditionJson, active	Approval workflow definition	1:N ApprovalStepDefinition
ApprovalStepDefinition	id, approvalDefinitionId, sequence, approverType, approverRef, minApprovals	Workflow step template	N:1 definition
ApprovalRequest	id, organizationId, subjectType, subjectId, definitionId, status, requestedById	Approval instance	1:N ApprovalStep
ApprovalStep	id, approvalRequestId, sequence, approverType, approverRef, status	Approval instance step	N:1 request
ApprovalAction	id, approvalStepId, actorId, action, comment, actedAt	Approval audit action	N:1 step
Document	id, organizationId, fileName, mimeType, size, bucket, objectKey, checksum, uploadedById	Object metadata	1:N versions/links
DocumentVersion	id, documentId, versionNo, objectKey, checksum, uploadedAt	File version	N:1 Document
DocumentLink	id, documentId, subjectType, subjectId, category	Generic attachment link	N:1 Document

Entity	Key fields	Purpose	Major relationships
Notification	id, organizationId, userId, type, title, body, readAt, createdAt	In-app notification	N:1 User
EmailOutbox	id, organizationId, template, recipient, payloadJson, status, attempts	Reliable email dispatch	Processed by worker
WebhookEndpoint	id, organizationId, url, secretEncrypted, eventsJson, active	External webhook config	1:N delivery
WebhookDelivery	id, endpointId, eventId, statusCode, attempts, deliveredAt	Webhook attempt log	N:1 endpoint
AuditLog	id, organizationId, actorUserId, action, subjectType, subjectId, beforeJson, afterJson, ip, createdAt	Immutable business audit	Tenant/system trace
BusinessEvent	id, organizationId, type, aggregateType, aggregateId, payloadJson, occurredAt, publishedAt	Outbox/event record	Feeds jobs/webhooks
CustomFieldDefinition	id, organizationId, subjectType, key, label, dataType, optionsJson	Tenant custom field	1:N values
CustomFieldValue	id, definitionId, subjectId, valueJson	Custom value	N:1 definition
BusinessRule	id, organizationId, triggerType, conditionJson, actionJson, active	Deterministic rule config	Evaluated by services/jobs
IdempotencyKey	id, organizationId, key, route, requestHash, responseJson, expiresAt	Duplicate mutation guard	Used by selected POST commands

Advanced Operations

Entity	Key fields	Purpose	Major relationships
Vehicle	id, organizationId, branchId, registrationNo, make, model, status	Fleet asset	1:N maintenance/fuel logs
VehicleMaintenance	id, vehicleId, serviceDate, cost, odometer, notes	Vehicle service history	N:1 Vehicle
FuelLog	id, vehicleId, employeeId, date, liters, cost, odometer	Fuel tracking	N:1 Vehicle
Tool	id, organizationId, toolNo, name, serialNo, status	Reusable field tool	1:N assignments
ToolAssignment	id, toolId, employeeId, issuedAt, dueAt, returnedAt	Tool custody	N:1 Tool/Employee
SafetyIncident	id, organizationId, projectId, siteld, reportedById, severity, status	Safety incident	1:N corrective actions
CorrectiveAction	id, safetyIncidentId, ownerId, dueDate, status, action	Safety corrective action	N:1 Incident
InspectionTemplate	id, organizationId, subjectType, name, version	Quality checklist template	1:N items/runs
InspectionRun	id, templateId, subjectType, subjectId, inspectorId, result, completedAt	Inspection execution	1:N result items

7.1 Core relationship matrix

Parent	Child	Cardinality	Mapping	Meaning
Organization	Branch	1:N	organizationId	Tenant has branches
Branch	Department	1:N	branchId	Branch structure
User	OrganizationMembership	1:N	userId	One login can join tenants
OrganizationMembership	Role	M:N	UserRole	Tenant-scoped RBAC
Customer	CustomerSite	1:N	customerId	Customer locations
Customer	Project	1:N	customerId	Customer projects
Customer	Contract	1:N	customerId	Commercial contracts
Customer	CustomerInvoice	1:N	customerId	Receivables
Opportunity	Quotation	1:N	opportunityId	Sales offers
Quotation	SalesOrder	1:0..1	quotationId	Accepted quote conversion
SalesOrder	Contract	1:0..1	salesOrderId	Commercial agreement
Contract	Project	1:N	contractId	Delivery projects
Vendor	SupplierQuotation	1:N	vendorId	RFQ responses
Vendor	PurchaseOrder	1:N	vendorId	Purchase commitments
PurchaseRequest	RFQ	1:0..N	purchaseRequestId	Sourcing
RFQ	SupplierQuotation	1:N	rfqId	Supplier bids
SupplierQuotation	PurchaseOrder	1:0..N	supplierQuotationId	Selected quote to PO
PurchaseOrder	GoodsReceipt	1:N	purchaseOrderId	Partial receipts
Product	StockTransaction	1:N	productId	Stock ledger
Warehouse	StockTransaction	1:N	warehouseId	Warehouse ledger
Product	SerialNumber	1:N	productId	Serialized units
Project	StockReservation	1:N	projectId	Project-reserved stock
Project	BillOfMaterials	1:N	projectId	Versioned material plan

Parent	Child	Cardinality	Mapping	Meaning
Project	ProjectTask	1:N	projectId	Project work
Project	Asset	1:N	projectId	Installed project assets
CustomerSite	Asset	1:N	siteId	Physical asset location
Asset	Ticket	1:N	assetId	Asset incidents
Ticket	WorkOrder	1:N	ticketId	Field jobs
WorkOrder	ServiceReport	1:N	workOrderId	Service evidence
Asset	MaintenancePlan	1:N	assetId	Preventive maintenance
CustomerInvoice	CustomerInvoiceItem	1:N	invoiceId	Invoice lines
Payment	CustomerInvoice	M:N	PaymentAllocation	Payments can settle multiple invoices
JournalEntry	JournalLine	1:N	journalEntryId	Double-entry posting
Account	JournalLine	1:N	accountId	Chart-of-account posting
ApprovalDefinition	ApprovalStepDefinition	1:N	approvalDefinitionId	Workflow template
ApprovalRequest	ApprovalStep	1:N	approvalRequestId	Runtime approval
Document	DocumentLink	1:N	documentId	Attachments to business subjects
WebhookEndpoint	WebhookDelivery	1:N	endpointId	External integration delivery history

7.2 Prisma mapping conventions

```

model PurchaseOrder {
  id          String    @id @default(uuid()) @db.Uuid
  organizationId String  @db.Uuid
  vendorId    String    @db.Uuid
  poNo        String
  status       PurchaseOrderStatus
  total        Decimal  @db.Decimal(18, 2)
  createdAt   DateTime @default(now())
  updatedAt   DateTime @updatedAt

  organization Organization @relation(fields: [organizationId], references: [id])
  vendor        Vendor      @relation(fields: [vendorId], references: [id])
  items          PurchaseOrderItem[]
  receipts       GoodsReceipt[]

  @@unique([organizationId, poNo])
  @@index([organizationId, status, createdAt])
  @@index([organizationId, vendorId])
}

```

Prisma rule

Database model files are grouped by domain, but Prisma remains the persistence layer rather than the API contract. Mapping functions explicitly convert Prisma results to shared response contracts.

8. API Architecture and Conventions

Concern	Convention
Base path	/api/v1
Transport	HTTPS JSON REST; multipart only where direct API file upload is required
Content type	application/json
Authentication	Short-lived access credential + secure HttpOnly refresh/session cookie
Tenant context	Resolved from authenticated membership; client cannot arbitrarily override organizationId
Authorization	Permission key + branch/resource scope checks
IDs	UUID in API paths; human numbers are separate searchable fields
Pagination	page + pageSize for ERP grids; cursor for timelines/events where appropriate
Sorting	sort=field:asc,field2:desc allowlisted per endpoint
Filtering	Explicit query filters; no raw database expression input
Mutation status transitions	Command endpoints such as /submit, /approve, /receive, /post, /close
Idempotency	Idempotency-Key required/recommended for payment, GRN, invoice post and other retry-sensitive commands

Concern	Convention
Errors	Stable error code + message + details + requestId
Validation	Shared Zod contracts where browser-safe; server-only validation extends them for authorization/business constraints
API docs	OpenAPI generated from route schemas and published at controlled /docs endpoint
Versioning	Breaking contract changes create /v2 or controlled compatibility migration; not silent response changes

Successful single-resource response

```
{
  "data": { ... },
  "meta": { "requestId": "..." }
}
```

List response

```
{
  "data": [ ... ],
  "meta": { "page": 1, "pageSize": 25, "total": 241, "requestId": "..." }
}
```

Error response

```
{
  "error": {
    "code": "PURCHASE_REQUEST_INVALID_STATE",
    "message": "Purchase request cannot be approved from DRAFT state.",
    "details": { "currentStatus": "DRAFT" },
    "requestId": "req..."
  }
}
```

9. API Endpoint Catalog

The following catalog is the target REST surface. CRUD endpoints are provided for editable master data; workflow state changes use explicit command endpoints. Exact payload fields are defined by shared contracts.

Approvals

Method	Endpoint	Purpose	Permission	Notes
GET	/api/v1/approvals/inbox	Current approver inbox	approval.view	Resolved by role/user scope
GET	/api/v1/approvals/:id	Approval request details	approval.view	
POST	/api/v1/approvals/:id/approve	Approve current step	approval.act	Maker-checker enforced
POST	/api/v1/approvals/:id/reject	Reject current step	approval.act	Comment required
POST	/api/v1/approvals/:id/return	Return for correction	approval.act	
GET	/api/v1/approval-definitions	List workflow definitions	workflow.manage	
POST	/api/v1/approval-definitions	Create definition	workflow.manage	Configurable conditions/steps

Assets

Method	Endpoint	Purpose	Permission	Notes
GET	/api/v1/assets	List assets	asset.view	Paginated/filterable
GET	/api/v1/assets/:id	Get asset details	asset.view	Tenant scoped
POST	/api/v1/assets	Create asset	asset.create	Validated shared contract
PATCH	/api/v1/assets/:id	Update editable fields	asset.update	Status is not freely patchable
POST	/api/v1/assets/register-from-stock	Register asset from serial/stock	asset.create	Requires eligible serial
POST	/api/v1/assets/:id/install	Install asset	asset.install	Atomic stock/asset history
POST	/api/v1/assets/:id/replace	Replace asset	asset.replace	Links old/new assets
POST	/api/v1/assets/:id/retire	Retire asset	asset.retire	Approval may apply
GET	/api/v1/assets/:id/history	Asset lifecycle history	asset.view	
POST	/api/v1/assets/:id/qr/rotate	Rotate QR token	asset.manage_qr	Revokes prior token
GET	/api/v1/asset-qr/:token	Resolve QR to authorized asset view	Authenticated/portal	Token alone does not bypass authorization
POST	/api/v1/assets/:id/rma	Create RMA	asset.rma	

Audit

Method	Endpoint	Purpose	Permission	Notes
GET	/api/v1/audit-logs	Search audit trail	audit.view	Sensitive permission
GET	/api/v1/audit-logs/:id	Audit event details	audit.view	

CRM

Method	Endpoint	Purpose	Permission	Notes
GET	/api/v1/leads	List leads	lead.view	Paginated/filterable
GET	/api/v1/leads/:id	Get lead details	lead.view	Tenant scoped
POST	/api/v1/leads	Create lead	lead.create	Validated shared contract
PATCH	/api/v1/leads/:id	Update editable fields	lead.update	Status is not freely patchable
GET	/api/v1/opportunities	List opportunities	opportunity.view	Paginated/filterable
GET	/api/v1/opportunities/:id	Get opportunity details	opportunity.view	Tenant scoped
POST	/api/v1/opportunities	Create opportunity	opportunity.create	Validated shared contract
PATCH	/api/v1/opportunities/:id	Update editable fields	opportunity.update	Status is not freely patchable
GET	/api/v1/site-surveys	List site surveys	site_survey.view	Paginated/filterable
GET	/api/v1/site-surveys/:id	Get site survey details	site_survey.view	Tenant scoped
POST	/api/v1/site-surveys	Create site survey	site_survey.create	Validated shared contract
PATCH	/api/v1/site-surveys/:id	Update editable fields	site_survey.update	Status is not freely patchable
GET	/api/v1/quotations	List quotations	quotation.view	Paginated/filterable
GET	/api/v1/quotations/:id	Get quotation details	quotation.view	Tenant scoped
POST	/api/v1/quotations	Create quotation	quotation.create	Validated shared contract
PATCH	/api/v1/quotations/:id	Update editable fields	quotation.update	Status is not freely patchable
GET	/api/v1/contracts	List contracts	contract.view	Paginated/filterable
GET	/api/v1/contracts/:id	Get contract details	contract.view	Tenant scoped
POST	/api/v1/contracts	Create contract	contract.create	Validated shared contract
PATCH	/api/v1/contracts/:id	Update editable fields	contract.update	Status is not freely patchable
POST	/api/v1/leads/:id/qualify	Qualify lead	lead.update	May create opportunity
POST	/api/v1/quotations/:id/send	Mark/send quotation	quotation.send	Queues PDF/email
POST	/api/v1/quotations/:id/accept	Accept quotation	quotation.approve	May create sales order
POST	/api/v1/contracts/:id/activate	Activate contract	contract.activate	Validates dates/terms

Customers

Method	Endpoint	Purpose	Permission	Notes
GET	/api/v1/customers	List customers	customer.view	Paginated/filterable
GET	/api/v1/customers/:id	Get customer details	customer.view	Tenant scoped
POST	/api/v1/customers	Create customer	customer.create	Validated shared contract
PATCH	/api/v1/customers/:id	Update editable fields	customer.update	Status is not freely patchable
GET	/api/v1/customer-sites	List customer sites	customer_site.view	Paginated/filterable
GET	/api/v1/customer-sites/:id	Get customer site details	customer_site.view	Tenant scoped
POST	/api/v1/customer-sites	Create customer site	customer_site.create	Validated shared contract
PATCH	/api/v1/customer-sites/:id	Update editable fields	customer_site.update	Status is not freely patchable
GET	/api/v1/customers/:id/timeline	Customer activity timeline	customer.view	Aggregated read model
GET	/api/v1/customer-sites/:id/assets	List assets at site	asset.view	

Documents

Method	Endpoint	Purpose	Permission	Notes
GET	/api/v1/documents	List documents	document.view	Paginated/filterable
GET	/api/v1/documents/:id	Get document details	document.view	Tenant scoped
POST	/api/v1/documents	Create document	document.create	Validated shared contract
DELETE	/api/v1/documents/:id	Delete/archive resource	document.delete	Subject to retention rules
POST	/api/v1/documents/upload-intent	Request presigned upload	document.create	Validates size/type/category
POST	/api/v1/documents/complete-upload	Register completed upload	document.create	Checksum/object existence check
GET	/api/v1/documents/:id/download-url	Get presigned download URL	document.view	Short-lived and authorized
POST	/api/v1/documents/:id/versions	Upload new version	document.update	

Finance

Method	Endpoint	Purpose	Permission	Notes
GET	/api/v1/customer-invoices	List customer invoices	invoice.view	Paginated/filterable

Method	Endpoint	Purpose	Permission	Notes
GET	/api/v1/customer-invoices/:id	Get invoice details	invoice.view	Tenant scoped
POST	/api/v1/customer-invoices	Create invoice	invoice.create	Validated shared contract
PATCH	/api/v1/customer-invoices/:id	Update editable fields	invoice.update	Status is not freely patchable
GET	/api/v1/supplier-invoices	List supplier invoices	supplier_invoice.view	Paginated/filterable
GET	/api/v1/supplier-invoices/:id	Get supplier invoice details	supplier_invoice.view	Tenant scoped
POST	/api/v1/supplier-invoices	Create supplier invoice	supplier_invoice.create	Validated shared contract
PATCH	/api/v1/supplier-invoices/:id	Update editable fields	supplier_invoice.update	Status is not freely patchable
GET	/api/v1/expenses	List expenses	expense.view	Paginated/filterable
GET	/api/v1/expenses/:id	Get expense details	expense.view	Tenant scoped
POST	/api/v1/expenses	Create expense	expense.create	Validated shared contract
PATCH	/api/v1/expenses/:id	Update editable fields	expense.update	Status is not freely patchable
POST	/api/v1/customer-invoices/:id/submit	Submit invoice approval	invoice.submit	
POST	/api/v1/customer-invoices/:id/approve	Approve invoice	invoice.approve	
POST	/api/v1/customer-invoices/:id/post	Post invoice	invoice.post	May create journal entry
POST	/api/v1/customer-invoices/:id/send	Send invoice	invoice.send	PDF/email job
POST	/api/v1/customer-invoices/:id/cancel	Cancel/reverse invoice	invoice.cancel	No destructive delete after posting
POST	/api/v1/supplier-invoices/:id/match	Run three-way match	supplier_invoice.match	PO + GRN + invoice
POST	/api/v1/supplier-invoices/:id/approve	Approve supplier invoice	supplier_invoice.approve	
GET	/api/v1/payments	List payments	payment.view	
POST	/api/v1/payments	Record payment	payment.create	Idempotency key; allocations transactional
GET	/api/v1/accounts	Chart of accounts	account.view	
POST	/api/v1/journal-entries	Create manual journal	journal.create	Restricted
POST	/api/v1/journal-entries/:id/post	Post journal	journal.post	Immutable after posting; reverse instead
GET	/api/v1/finance/receivables	AR aging	finance.view	
GET	/api/v1/finance/payables	AP aging	finance.view	

HR

Method	Endpoint	Purpose	Permission	Notes
GET	/api/v1/employees	List employees	employee.view	Paginated/filterable
GET	/api/v1/employees/:id	Get employee details	employee.view	Tenant scoped
POST	/api/v1/employees	Create employee	employee.create	Validated shared contract
PATCH	/api/v1/employees/:id	Update editable fields	employee.update	Status is not freely patchable
GET	/api/v1/attendance	List attendance	attendance.view	Paginated/filterable
GET	/api/v1/attendance/:id	Get attendance details	attendance.view	Tenant scoped
GET	/api/v1/leave-requests	List leave requests	leave.view	Paginated/filterable
GET	/api/v1/leave-requests/:id	Get leave details	leave.view	Tenant scoped
POST	/api/v1/leave-requests	Create leave	leave.create	Validated shared contract
GET	/api/v1/payroll-runs	List payroll runs	payroll.view	Paginated/filterable
GET	/api/v1/payroll-runs/:id	Get payroll details	payroll.view	Tenant scoped
POST	/api/v1/payroll-runs	Create payroll	payroll.create	Validated shared contract
POST	/api/v1/leave-requests/:id/submit	Submit leave request	leave.submit	Creates approval request when required
POST	/api/v1/leave-requests/:id/cancel	Cancel leave request	leave.cancel	State transition
POST	/api/v1/payroll-runs/:id/calculate	Calculate payroll	payroll.calculate	Background-capable
POST	/api/v1/payroll-runs/:id/approve	Approve payroll	payroll.approve	Maker-checker
POST	/api/v1/payroll-runs/:id/post	Post payroll	payroll.post	Transactional finance integration

Identity

Method	Endpoint	Purpose	Permission	Notes
POST	/api/v1/auth/login	Authenticate user	Public	Rate limited; may return MFA challenge
POST	/api/v1/auth/mfa/verify	Complete MFA challenge	Public/Challenge	Rate limited
POST	/api/v1/auth/refresh	Rotate access session	Session cookie	Refresh token/session rotation
POST	/api/v1/auth/logout	Revoke current session	Authenticated	
POST	/api/v1/auth/logout-all	Revoke all sessions	Authenticated	
GET	/api/v1/auth/me	Current user/membership/permissions	Authenticated	
GET	/api/v1/auth/sessions	List active sessions	Authenticated	
DELETE	/api/v1/auth/sessions/:sessionId	Revoke session	Authenticated	

Inventory

Method	Endpoint	Purpose	Permission	Notes
GET	/api/v1/products	List products	product.view	Paginated/filterable
GET	/api/v1/products/:id	Get product details	product.view	Tenant scoped
POST	/api/v1/products	Create product	product.create	Validated shared contract
PATCH	/api/v1/products/:id	Update editable fields	product.update	Status is not freely patchable
GET	/api/v1/warehouses	List warehouses	warehouse.view	Paginated/filterable
GET	/api/v1/warehouses/:id	Get warehouse details	warehouse.view	Tenant scoped
POST	/api/v1/warehouses	Create warehouse	warehouse.create	Validated shared contract
PATCH	/api/v1/warehouses/:id	Update editable fields	warehouse.update	Status is not freely patchable
GET	/api/v1/inventory/stock	Current stock balances	inventory.view	Warehouse/product filters
GET	/api/v1/inventory/ledger	Stock ledger	inventory.view	Immutable transaction history
GET	/api/v1/inventory/serials/:serialNo	Lookup serial number	inventory.view	Global tenant search
POST	/api/v1/inventory/reservations	Reserve stock	inventory.reserve	Project/BOM reference
DELETE	/api/v1/inventory/reservations/:id	Release reservation	inventory.reserve	
POST	/api/v1/inventory/transfers	Create transfer	inventory.transfer	
POST	/api/v1/inventory/transfers/:id/dispatch	Dispatch transfer	inventory.transfer	Atomic source stock movement
POST	/api/v1/inventory/transfers/:id/receive	Receive transfer	inventory.receive	Atomic destination movement
POST	/api/v1/inventory/adjustments	Create adjustment	inventory.adjust	Approval threshold may apply
POST	/api/v1/inventory/adjustments/:id/post	Post adjustment	inventory.adjust	Creates immutable ledger

Maintenance

Method	Endpoint	Purpose	Permission	Notes
GET	/api/v1/maintenance/plans	List maintenance plans	maintenance.view	
POST	/api/v1/maintenance/plans	Create plan	maintenance.create	
GET	/api/v1/maintenance/schedule	Upcoming maintenance	maintenance.view	Date/branch/site filters
POST	/api/v1/maintenance/schedules/:id/generate-work-order	Generate work order	maintenance.execute	Idempotent
POST	/api/v1/maintenance/executions/:id/complete	Complete maintenance	maintenance.execute	Updates next due date

Notifications

Method	Endpoint	Purpose	Permission	Notes
GET	/api/v1/notifications	List notifications	Authenticated	
POST	/api/v1/notifications/:id/read	Mark read	Authenticated	
POST	/api/v1/notifications/read-all	Mark all read	Authenticated	

Organization

Method	Endpoint	Purpose	Permission	Notes
GET	/api/v1/branches	List branches	branch.view	Paginated/filterable
GET	/api/v1/branches/:id	Get branch details	branch.view	Tenant scoped
POST	/api/v1/branches	Create branch	branch.create	Validated shared contract
PATCH	/api/v1/branches/:id	Update editable fields	branch.update	Status is not freely patchable
GET	/api/v1/departments	List departments	department.view	Paginated/filterable
GET	/api/v1/departments/:id	Get department details	department.view	Tenant scoped
POST	/api/v1/departments	Create department	department.create	Validated shared contract
PATCH	/api/v1/departments/:id	Update editable fields	department.update	Status is not freely patchable

Platform

Method	Endpoint	Purpose	Permission	Notes
GET	/api/v1/search	Global search	Authenticated	Permission-filtered results
GET	/api/v1/calendar	Unified calendar	Authenticated	Permission-filtered
GET	/api/v1/health/ready	Readiness probe	Internal/Public controlled	No secrets

Method	Endpoint	Purpose	Permission	Notes
GET	/api/v1/health/live	Liveness probe	Internal/Public controlled	

Procurement

Method	Endpoint	Purpose	Permission	Notes
GET	/api/v1/purchase-requests	List purchase requests	purchase_request.view	
POST	/api/v1/purchase-requests	Create purchase request	purchase_request.create	
GET	/api/v1/purchase-requests/:id	Get purchase request	purchase_request.view	
PATCH	/api/v1/purchase-requests/:id	Edit draft request	purchase_request.update	DRAFT only
POST	/api/v1/purchase-requests/:id/submit	Submit for approval	purchase_request.submit	Creates approval workflow
POST	/api/v1/purchase-requests/:id/approve	Approve purchase request	purchase_request.approve	Explicit command
POST	/api/v1/purchase-requests/:id/reject	Reject purchase request	purchase_request.approve	Comment required
POST	/api/v1/purchase-requests/:id/create-rfq	Create RFQ	rfq.create	Requires approved PR
GET	/api/v1/rfqs	List RFQs	rfq.view	
POST	/api/v1/rfqs	Create RFQ	rfq.create	
POST	/api/v1/rfqs/:id/invite-vendors	Invite vendors	rfq.update	
POST	/api/v1/rfqs/:id/publish	Publish RFQ	rfq.publish	
POST	/api/v1/rfqs/:id/close	Close RFQ	rfq.close	
GET	/api/v1/rfqs/:id/comparison	Quotation comparison	supplier_quotation.view	Deterministic comparison view
POST	/api/v1/supplier-quotations	Record supplier quotation	supplier_quotation.create	Vendor portal/internal
POST	/api/v1/supplier-quotations/:id/select	Select quotation	supplier_quotation.select	Approval may be required
GET	/api/v1/purchase-orders	List purchase orders	purchase_order.view	
POST	/api/v1/purchase-orders	Create purchase order	purchase_order.create	
POST	/api/v1/purchase-orders/:id/submit	Submit PO approval	purchase_order.submit	
POST	/api/v1/purchase-orders/:id/approve	Approve PO	purchase_order.approve	Maker-checker
POST	/api/v1/purchase-orders/:id/send	Send PO to vendor	purchase_order.send	Queues document/email
POST	/api/v1/purchase-orders/:id/cancel	Cancel PO	purchase_order.cancel	Checks receipt state
GET	/api/v1/goods-receipts	List GRNs	goods_receipt.view	
POST	/api/v1/goods-receipts	Receive goods	goods_receipt.create	Atomic GRN + inventory transaction
GET	/api/v1/goods-receipts/:id	GRN details	goods_receipt.view	
POST	/api/v1/goods-receipts/:id/inspect	Record quality inspection	goods_receipt.inspect	

Projects

Method	Endpoint	Purpose	Permission	Notes
GET	/api/v1/projects	List projects	project.view	Paginated/filterable
GET	/api/v1/projects/:id	Get project details	project.view	Tenant scoped
POST	/api/v1/projects	Create project	project.create	Validated shared contract
PATCH	/api/v1/projects/:id	Update editable fields	project.update	Status is not freely patchable
GET	/api/v1/project-tasks	List project tasks	project_task.view	Paginated/filterable
GET	/api/v1/project-tasks/:id	Get project task details	project_task.view	Tenant scoped
POST	/api/v1/project-tasks	Create project task	project_task.create	Validated shared contract
PATCH	/api/v1/project-tasks/:id	Update editable fields	project_task.update	Status is not freely patchable
GET	/api/v1/projects/:id/bom	Project BOM	project.view	
PUT	/api/v1/projects/:id/bom	Create/update draft BOM	project.update	Versioned
POST	/api/v1/projects/:id/bom/:bomId/approve	Approve BOM	project.approve	
GET	/api/v1/projects/:id/budget	Project budget	project.view	
POST	/api/v1/projects/:id/material-request	Create material requirement	project.update	Links procurement/inventory
GET	/api/v1/projects/:id/costing	Project cost and profitability	project.view_financials	Aggregated read model
POST	/api/v1/projects/:id/handover	Complete handover	project.handover	Customer acceptance evidence
GET	/api/v1/projects/:id/timeline	Project activity timeline	project.view	

Reports

Method	Endpoint	Purpose	Permission	Notes
GET	/api/v1/reports	List reports	report.view	Paginated/filterable
GET	/api/v1/reports/:id	Get report details	report.view	Tenant scoped
POST	/api/v1/reports/exports	Request report export	report.export	Returns async job id
GET	/api/v1/reports/exports/:jobId	Check/download export	report.export	

Service

Method	Endpoint	Purpose	Permission	Notes
GET	/api/v1/tickets	List tickets	ticket.view	Paginated/filterable
GET	/api/v1/tickets/:id	Get ticket details	ticket.view	Tenant scoped
POST	/api/v1/tickets	Create ticket	ticket.create	Validated shared contract
PATCH	/api/v1/tickets/:id	Update editable fields	ticket.update	Status is not freely patchable
GET	/api/v1/work-orders	List work orders	workorder.view	Paginated/filterable
GET	/api/v1/work-orders/:id	Get workorder details	workorder.view	Tenant scoped
POST	/api/v1/work-orders	Create workorder	workorder.create	Validated shared contract
PATCH	/api/v1/work-orders/:id	Update editable fields	workorder.update	Status is not freely patchable
POST	/api/v1/tickets/:id/assign	Assign ticket	ticket.assign	
POST	/api/v1/tickets/:id/resolve	Resolve ticket	ticket.resolve	
POST	/api/v1/tickets/:id/close	Close ticket	ticket.close	Customer confirmation rules
POST	/api/v1/work-orders/:id/assign	Assign technician	workorder.assign	
POST	/api/v1/work-orders/:id/accept	Technician accepts job	workorder.accept	Assigned technician only
POST	/api/v1/work-orders/:id/start-travel	Start travel	workorder.update	
POST	/api/v1/work-orders/:id/arrive	Mark onsite	workorder.update	
POST	/api/v1/work-orders/:id/start	Start work	workorder.update	
POST	/api/v1/work-orders/:id/complete	Complete work order	workorder.close	Requires service report
POST	/api/v1/work-orders/:id/service-report	Create service report	workorder.update	Parts consumption transaction

Vendors

Method	Endpoint	Purpose	Permission	Notes
GET	/api/v1/vendors	List vendors	vendor.view	Paginated/filterable
GET	/api/v1/vendors/:id	Get vendor details	vendor.view	Tenant scoped
POST	/api/v1/vendors	Create vendor	vendor.create	Validated shared contract
PATCH	/api/v1/vendors/:id	Update editable fields	vendor.update	Status is not freely patchable
GET	/api/v1/vendors/:id/performance	Vendor performance metrics	vendor.view	
GET	/api/v1/vendors/:id/purchase-orders	Vendor POs	purchase_order.view	

10. API Contracts and Example Payloads

Shared contracts live under shared/src/contracts and are imported by both frontend and backend. The backend may apply stricter server-only authorization and persistence validation after the shared structural contract passes.

```
shared/src/contracts/customers/create-customer.contract.ts
```

```
export const CreateCustomerSchema = z.object({
  code: z.string().min(2).max(30),
  name: z.string().min(2).max(200),
  taxNo: z.string().max(80).optional(),
  primaryContact: z.object({
    name: z.string().min(2),
    email: z.string().email().optional(),
    phone: z.string().min(7).optional()
  }).optional()
});

export type CreateCustomerInput = z.infer<typeof CreateCustomerSchema>;
```

Login

POST /auth/login

Request

```
{
  "email": "manager@example.com",
  "password": "*****"
}
```

Response

```
{
  "data": {
    "accessToken": "short-lived-token",
    "requiresMfa": false,
    "user": { "id": "uuid", "name": "Ahmed", "memberships": [...] }
  }
}
```

Create purchase request

POST /purchase-requests

Request

```
{
  "projectId": "uuid",
  "requiredDate": "2026-09-20",
  "reason": "Project material shortage",
  "items": [
    { "productId": "uuid", "quantity": "35", "estimatedUnitPrice": "18500.00" }
  ]
}
```

Response

```
{
  "data": {
    "id": "uuid",
    "prNo": "PR-2026-0031",
    "status": "DRAFT",
    "items": [...]
  }
}
```

Approve purchase request

POST /purchase-requests/:id/approve

Request

```
{
  "comment": "Budget verified and material is required."
}
```

Response

```
{
  "data": {
    "id": "uuid",
    "status": "APPROVED",
    "approval": { "currentStep": null, "completed": true }
  }
}
```

Receive goods

POST /goods-receipts

Request

```
{
  "purchaseOrderId": "uuid",
  "warehouseId": "uuid",
  "receivedAt": "2026-09-18T09:30:00Z",
  "items": [
    {
      "purchaseOrderItemId": "uuid",
      "receivedQty": "10",
      "acceptedQty": "9",
      "damagedQty": "1",
      "serialNumbers": ["SN1001", "SN1002"]
    }
  ]
}
```

Response

```
{
  "data": {
    "id": "uuid",
    "grnNo": "GRN-2026-0082",
    "status": "RECEIVED",
    "stockTransactions": ["uuid", "uuid"]
  }
}
```

Stock transfer**POST /inventory/transfers****Request**

```
{
  "fromWarehouseId": "uuid",
  "toWarehouseId": "uuid",
  "items": [{ "productId": "uuid", "quantity": "5" }]
}
```

Response

```
{
  "data": { "id": "uuid", "transferNo": "TRF-2026-0044", "status": "DRAFT" }
}
```

Create project**POST /projects****Request**

```
{
  "customerId": "uuid",
  "contractId": "uuid",
  "siteId": "uuid",
  "name": "Campus CCTV Upgrade",
  "managerId": "uuid",
  "startDate": "2026-09-10",
  "dueDate": "2026-11-15",
  "contractValue": "5000000.00"
}
```

Response

```
{
  "data": { "id": "uuid", "projectNo": "PRJ-2026-0018", "status": "DRAFT" }
}
```

Install asset

POST /assets/:id/install

Request

```
{
  "siteId": "uuid",
  "areaId": "uuid",
  "projectId": "uuid",
  "technicianId": "uuid",
  "installedAt": "2026-09-20T11:10:00Z",
  "locationText": "Building A - Floor 2 - Corridor"
}
```

Response

```
{
  "data": {
    "id": "uuid",
    "assetNo": "AST-000145",
    "status": "ACTIVE",
    "qr": { "token": "opaque-token" }
  }
}
```

Create ticket

POST /tickets

Request

```
{
  "customerId": "uuid",
  "siteId": "uuid",
  "assetId": "uuid",
  "category": "EQUIPMENT_FAILURE",
  "priority": "HIGH",
  "subject": "Camera offline",
  "description": "Camera stopped responding this morning."
}
```

Response

```
{
  "data": { "id": "uuid", "ticketNo": "TKT-2026-0188", "status": "OPEN", "sla": {...} }
}
```

Complete work order

POST /work-orders/:id/complete

Request

```
{
  "serviceReportId": "uuid",
  "customerConfirmed": true
}
```

Response

```
{
  "data": { "id": "uuid", "status": "CLOSED", "closedAt": "..." }
}
```

Create customer invoice

POST /customer-invoices

Request

```
{
  "customerId": "uuid",
  "projectId": "uuid",
  "issueDate": "2026-10-01",
  "dueDate": "2026-10-31",
  "items": [
    { "description": "Milestone 1", "quantity": "1", "unitPrice": "1500000.00", "taxRate": "0" }
  ]
}
```

Response

```
{
  "data": { "id": "uuid", "invoiceNo": "INV-2026-0091", "status": "DRAFT", "total": "1500000.00" }
}
```

Record payment**POST /payments****Request**

```
{
  "direction": "INBOUND",
  "partyType": "CUSTOMER",
  "partyId": "uuid",
  "amount": "500000.00",
  "method": "BANK_TRANSFER",
  "paidAt": "2026-10-10T10:00:00Z",
  "allocations": [{ "invoiceId": "uuid", "amount": "500000.00" }]
}
```

Response

```
{
  "data": { "id": "uuid", "paymentNo": "PAY-2026-0033", "status": "POSTED" }
}
```

Presigned file upload**POST /documents/upload-intent****Request**

```
{
  "fileName": "site-photo.jpg",
  "mimeType": "image/jpeg",
  "size": 842113,
  "subjectType": "PROJECT",
  "subjectId": "uuid",
  "category": "SITE_PHOTO"
}
```

Response

```
{
  "data": {
    "uploadUrl": "https://minio/...signed...",
    "objectKey": "organizations/org-id/projects/project-id/site-photos/file-id.jpg",
    "expiresInSeconds": 300
  }
}
```

11. Authentication, Sessions, RBAC and Tenant Isolation

```
Login -> Password verification -> Optional MFA -> Session record
      -> Short-lived access credential
      -> Secure HttpOnly refresh/session cookie
```

Each request -> authenticate -> resolve membership -> tenant context
 -> permission check -> branch/resource scope -> service

Control	Implementation
Password storage	Argon2id or bcrypt with project-standard parameters; never reversible encryption.
Session model	Server-side revocable session with hashed refresh token/session secret.
MFA	TOTP for privileged roles initially; recovery codes stored hashed.
Access token	Short lifetime; carries identity/session reference, not authoritative permission lists that can become stale.
Permission model	role -> permissions; membership -> roles. Permission keys are canonical shared constants.
Resource scope	Permission alone is insufficient; services check tenant, branch, ownership and assignment scope.
Maker-checker	High-risk creator cannot be sole approver of the same transaction.
Tenant isolation	organizationId comes from authenticated membership/context and is injected into every tenant repository query.
Portal users	Customer/vendor portal identities are restricted to their linked customer/vendor records.
Admin actions	Role/permission/session changes are audited.

11.1 Permission catalog

Permission key	Purpose
identity.user.view	View tenant users
identity.user.manage	Create/update/suspend users
identity.role.manage	Manage roles and permissions
organization.view	View organization structure
organization.manage	Manage branches/departments
employee.view	View employees
employee.manage	Manage employee records
attendance.manage	Manage attendance
leave.approve	Approve leave
payroll.view	View payroll
payroll.approve	Approve payroll
customer.view	View customers
customer.create	Create customers
customer.update	Update customers
lead.view	View leads
lead.create	Create leads
quotation.create	Create quotations
quotation.approve	Approve/accept quotations
contract.manage	Manage contracts
vendor.view	View vendors
vendor.manage	Manage vendors
purchase_request.create	Create PR
purchase_request.approve	Approve PR
rfq.create	Create RFQ
supplier_quotation.select	Select supplier quotation
purchase_order.create	Create PO
purchase_order.approve	Approve PO
goods_receipt.create	Receive goods
inventory.view	View stock
inventory.reserve	Reserve stock
inventory.transfer	Transfer stock
inventory.receive	Receive transfer
inventory.adjust	Create/post adjustments
project.view	View projects
project.create	Create projects
project.update	Update projects
project.approve	Approve project controls/BOM
project.view_financials	View project costing
project.handover	Complete handover
asset.view	View assets
asset.create	Register assets
asset.install	Install assets
asset.replace	Replace assets
asset.retire	Retire assets

Permission key	Purpose
asset.rma	Manage RMA
asset.manage_qr	Rotate/manage QR tokens
ticket.view	View tickets
ticket.create	Create tickets
ticket.assign	Assign tickets
ticket.resolve	Resolve tickets
workorder.view	View work orders
workorder.assign	Assign technicians
workorder.update	Update field status
workorder.close	Close jobs
maintenance.view	View maintenance
maintenance.create	Create plans
maintenance.execute	Execute maintenance
invoice.view	View customer invoices
invoice.create	Create invoices
invoice.approve	Approve invoices
invoice.post	Post invoices
invoice.cancel	Cancel/reverse invoices
supplier_invoice.approve	Approve AP invoices
payment.view	View payments
payment.create	Record payments
expense.approve	Approve expenses
account.view	View chart of accounts
journal.create	Create journals
journal.post	Post journals
finance.view	View AR/AP reports
approval.view	View approval inbox
approval.act	Approve/reject/return
workflow.manage	Configure approval workflows
document.view	View documents
document.create	Upload documents
document.delete	Delete/archive documents
report.view	View reports
report.export	Export reports
audit.view	View audit logs

12. MinIO File Storage Design

Storage boundary

Business modules never call the MinIO SDK directly. They call a backend StorageService. This keeps object storage replaceable and centralizes authorization, naming, checksum, content-type and retention rules.

```

MinIO buckets
├── nexora-private
│   └── organizations/{organizationId}/
│       ├── employees/{employeeId}/...
│       ├── customers/{customerId}/...
│       ├── vendors/{vendorId}/...
│       ├── projects/{projectId}/...
│       ├── assets/{assetId}/...
│       ├── work-orders/{workOrderId}/...
│       └── finance/{invoiceId}/...
└── nexora-public      # only intentionally public brand/static resources

```

File operation	Flow
Small upload	Browser -> Fastify multipart -> validation -> StorageService -> MinIO -> Document metadata transaction
Large/direct upload	Browser requests upload intent -> Fastify authorizes -> short-lived presigned PUT -> browser uploads to MinIO -> complete-upload verifies object/checksum -> Document record
Download	Browser -> API authorization -> presigned GET (short expiry) -> MinIO
New version	Upload new object key -> DocumentVersion append -> previous version retained according to policy
Deletion	Business retention check -> metadata state change -> object deletion/retention job where allowed

13. Redis, BullMQ and Eventing

Queue / job	Producer	Worker action	Retry / idempotency
document.generate.invoice	Finance	Generate PDF, store in MinIO, link Document	Idempotent by invoice version
email.send	Any domain	Render template and send email	Retry with capped exponential backoff
notification.create	Any domain	Create notification/fan-out	Deduplicate by event + recipient
maintenance.scan	Scheduler	Find due schedules, create work-order jobs	Unique schedule occurrence
contract.expiry.scan	Scheduler	Generate expiry notifications	Date + contract dedupe
invoice.overdue.scan	Scheduler	Mark eligible invoices overdue	Transactional status guard
report.export	Reports	Generate CSV/XLSX/PDF and store file	Job id + parameters hash
webhook.deliver	Eventing	Sign and POST webhook payload	Delivery record + retry policy
document.scan	Documents	Optional malware/validation pipeline	Object checksum key

Eventing rule

Use events/queues for email, notifications, document generation, analytics and integrations. Do not move stock, money or approval state asynchronously when those changes are part of the same business transaction.

14. Security Architecture

Area	Required control
Authentication	Strong password hashing, MFA for privileged users, lockout/rate limiting, session revocation
Authorization	RBAC plus resource/branch/tenant scope checks inside services
Tenant isolation	organizationId enforced server-side; test cross-tenant access as a dedicated security suite
Input validation	Zod/route schema validation, allowlisted filters/sorts, bounded pagination
SQL injection	Prisma parameterization; raw SQL only reviewed and parameterized
XSS	React escaping, output encoding, CSP/security headers, sanitize rich text inputs if introduced
CSRF	Use SameSite/CSRF controls for cookie-authenticated mutation endpoints
File uploads	MIME/extension checks, size limits, content sniffing, private buckets, optional malware scanning
Secrets	Environment/secret manager; never shared package or client bundle
Audit	Business audit logs for high-risk actions; log before/after summary without sensitive secrets
PII	Field-level minimization; encryption for selected secrets; access logs for sensitive records
Rate limiting	Login, MFA, password reset, public/portal and high-cost endpoints
Idempotency	Payments, GRNs and retry-sensitive commands protected against duplicate submission
Headers/TLS	Nginx TLS, HSTS when appropriate, CSP, frame/content-type/referrer policies
Dependency security	Lockfile, Dependabot/Renovate, npm/pnpm audit, CodeQL/Semgrep in CI
Backups	Encrypted backups, restore test, retention policy, restricted access

15. Logging, Audit, Observability and Reporting

Telemetry	Purpose	Examples
Request log	Operational HTTP diagnostics	requestId, route, status, duration, userId, organizationId; never passwords/tokens
Business audit	Compliance and traceability	PO_APPROVED, STOCK_ADJUSTED, PAYMENT_POSTED, ROLE_CHANGED
Domain event	Async integration/outbox	invoice.posted, asset.installed, ticket.created
Worker log	Background job reliability	jobId, queue, attempt, duration, failure reason
Metrics	Service health	request latency, 5xx rate, DB pool, Redis, queue depth, failed jobs

Telemetry	Purpose	Examples
Alerts	Actionable incidents	API unavailable, repeated worker failures, backup failure, queue backlog
Reports	Business intelligence	project profitability, stock value, AR/AP aging, SLA compliance

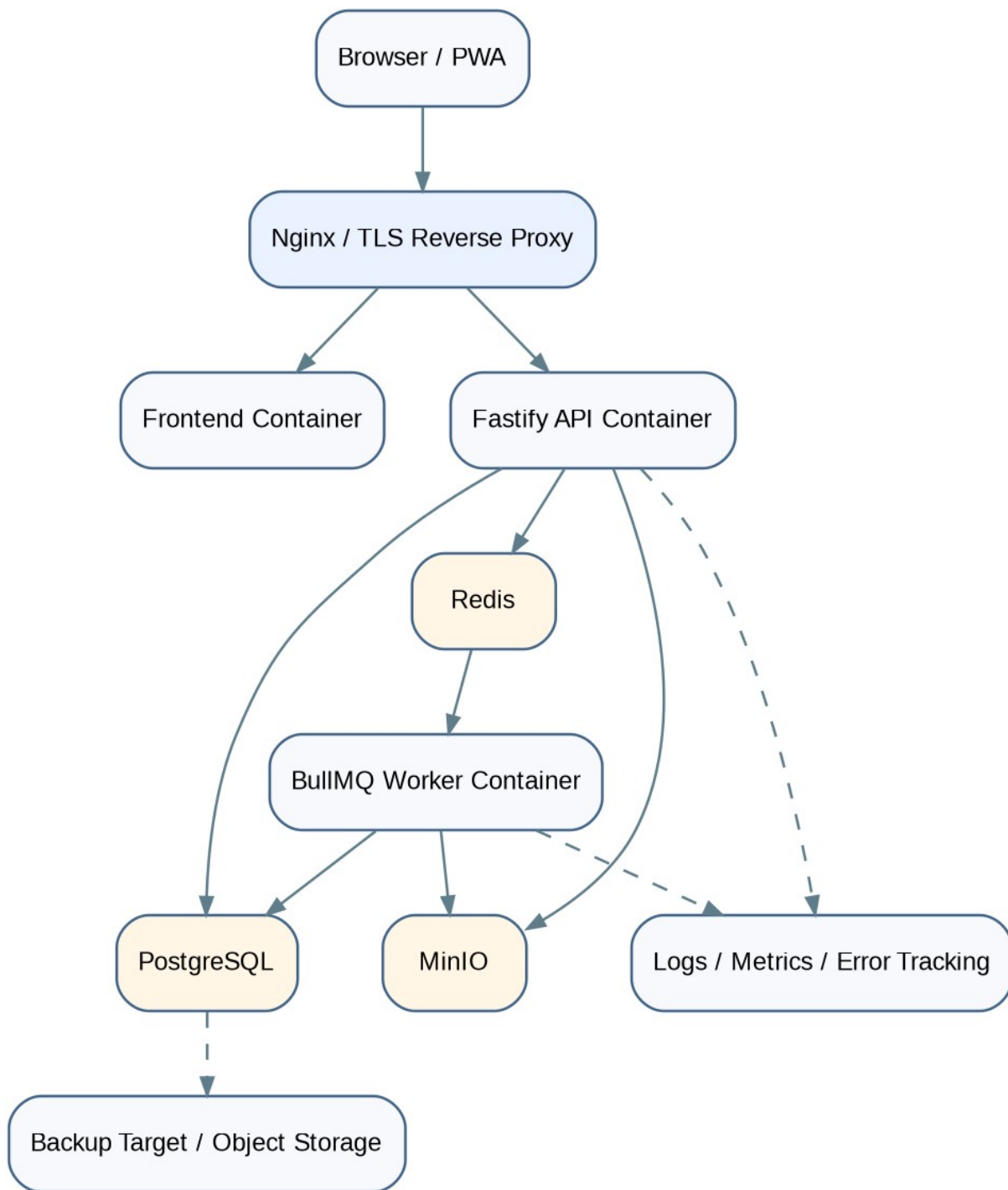
16. Testing Strategy

Test layer	Scope	Examples
Unit	Pure business rules/services	Approval threshold, invoice totals, status transition, SLA calculation
Repository integration	Prisma + test PostgreSQL	Tenant filtering, unique constraints, transaction behavior
API integration	Fastify inject against test DB	Authentication, permission denial, request validation, command status codes
Workflow integration	Cross-module business flows	PR -> approval -> RFQ -> PO -> GRN -> stock; ticket -> work order -> parts -> close
E2E	Browser against full stack	Login, create project, receive stock, install asset, invoice/payment
Security	Abuse/authorization	Cross-tenant IDOR, privilege escalation, upload abuse, rate limit
Migration	Schema evolution	Migrate clean DB and representative previous snapshot
Performance	Critical grids/commands	Stock ledger filters, global search, concurrent receipt/transfer operations
Backup/restore	Operational recovery	Restore PostgreSQL + object metadata and verify file references

16.1 Critical workflow test cases

- Receiving the same purchase order line concurrently cannot over-receive beyond configured tolerance.
- Two projects cannot reserve more free stock than exists.
- A user from Organization A cannot query, mutate or presign a document belonging to Organization B.
- A purchase-request creator cannot self-approve when maker-checker applies.
- Posting a payment twice with the same idempotency key produces one financial effect.
- Posted journal entries and stock ledger records are reversed, not silently edited/deleted.
- Closing a work order with consumed parts creates matching stock transactions.
- Asset installation links the correct serialized unit and prevents the serial from remaining available stock.
- Invoice status and balance reconcile with allocations after partial and full payments.
- Expired/rotated asset QR tokens do not grant access.

17. Docker, Deployment, CI/CD and Infrastructure



Reference deployment topology

```

docker compose services
├─ web          # Next.js
├─ api          # Fastify
├─ worker       # BullMQ processors/schedulers
├─ postgres
├─ redis
├─ minio
└─ nginx
  
```

Pipeline stage	Checks/actions
Install	pnpm install --frozen-lockfile
Static quality	ESLint, Prettier check, TypeScript typecheck
Tests	Unit + repository/API integration
Security	Dependency audit, CodeQL/Semgrep where configured
Database	Validate Prisma schema; migration safety review
Build	Build frontend, backend and worker artifacts/containers
Container checks	Image scan and non-root runtime where practical
Deploy	Apply migrations using controlled job, deploy containers, readiness checks
Post-deploy	Smoke tests, health checks, rollback path

17.1 Environment separation

Environment	Purpose	Data policy
Local	Developer workstation	Synthetic/local only
Test	Automated integration tests	Ephemeral synthetic data
Development	Shared integration environment	Non-production data
Staging	Release candidate verification	Production-like configuration; sanitized data only
Production	Live service	Strict access, backup, audit and monitoring

18. Development Roadmap and Definition of Done

Phase	Theme	Deliverables
Phase 0	Repository foundation	pnpm workspace, shared contracts, Fastify bootstrap, Prisma, Docker, lint/test/CI
Phase 1	Identity & organization	Users, sessions, MFA baseline, memberships, roles, permissions, branches, departments, audit
Phase 2	Business masters	Employees, customers, sites, vendors, products, warehouses
Phase 3	Inventory	Balances, immutable ledger, serial/batch, transfer, reservation, adjustment
Phase 4	Procurement	PR, approval, RFQ, quotation comparison, PO, GRN and receiving transactions
Phase 5	Projects	Projects, tasks, milestones, BOM, budgets, material requirement, costing read model
Phase 6	Assets	Serialized asset registration, installation, QR, warranty, RMA, history
Phase 7	Field service	Tickets, SLA, work orders, technician assignments, service reports, parts consumption
Phase 8	Maintenance	Plans, schedules, generated work orders, execution, reminders
Phase 9	Finance	AR/AP invoice, payments, expenses, three-way match, basic GL/postings
Phase 10	Enterprise controls	Configurable approvals, documents, notifications, rules, reports, customer/vendor portals
Phase 11	Advanced operations	Fleet, tools, safety, quality, localization, offline PWA, integrations

Definition of Done for a module

A module is not done when CRUD works. It is done when permissions, tenant filtering, validation, state transitions, audit, transaction handling, tests, API documentation, error codes, migration/indexes and operational logging are complete.

Appendix A - Canonical Status Models

PurchaseRequest statuses
DRAFT
SUBMITTED
UNDER_REVIEW
APPROVED
REJECTED
CONVERTED_TO_RFQ
CANCELLED

RFQ statuses
DRAFT
PUBLISHED
OPEN
CLOSED
AWARDED
CANCELLED

PurchaseOrder statuses
DRAFT
APPROVAL_PENDING
APPROVED
SENT
PARTIALLY_RECEIVED
RECEIVED
CLOSED
CANCELLED

GoodsReceipt statuses
DRAFT
RECEIVED
INSPECTION_PENDING
ACCEPTED
PARTIALLY_ACCEPTED
REJECTED

Project statuses
DRAFT
PLANNED
ACTIVE
ON_HOLD
COMPLETED
HANDED_OVER
CANCELLED

Asset statuses
PROCURED
IN_WAREHOUSE
ALLOCATED
ISSUED
INSTALLED
ACTIVE
UNDER_MAINTENANCE
REPAIRED
REPLACED
RETIRED

Ticket statuses
OPEN
ASSIGNED
IN_PROGRESS

Ticket statuses
WAITING_CUSTOMER
WAITING_VENDOR
RESOLVED
CLOSED
CANCELLED

WorkOrder statuses
NEW
VALIDATED
ASSIGNED
TECHNICIAN_ACCEPTED
TRAVELLING
ON_SITE
DIAGNOSIS
WORK_IN_PROGRESS
WAITING_FOR_PART
RESOLVED
CUSTOMER_CONFIRMATION
CLOSED
CANCELLED

Invoice statuses
DRAFT
APPROVAL_PENDING
APPROVED
POSTED
SENT
PARTIALLY_PAID
PAID
OVERDUE
CANCELLED
REVERSED

ApprovalRequest statuses
PENDING
IN_PROGRESS
APPROVED
REJECTED
RETURNED
CANCELLED

Contract statuses
DRAFT
UNDER_REVIEW
ACTIVE
EXPIRING
EXPIRED
TERMINATED

LeaveRequest statuses
DRAFT
SUBMITTED
APPROVED
REJECTED
CANCELLED

Appendix B - Domain Event Catalog

Event	Owner	Typical consumers
identity.user.created	Identity	Notification/audit
identity.session.revoked	Identity	Security notification

Event	Owner	Typical consumers
purchase_request.submitted	Procurement	Approval workflow
purchase_request.approved	Procurement	RFQ eligibility/notification
purchase_order.approved	Procurement	Vendor notification/webhook
goods_receipt.received	Procurement	Notifications/reporting
stock.low	Inventory	Replenishment notification
stock.transfer.received	Inventory	Project/warehouse notification
project.created	Projects	Notification/webhook
project.handed_over	Projects	Invoice/service eligibility
asset.installed	Assets	QR/document/customer notification
asset.warranty.expiring	Assets	Notification
ticket.created	Service	Assignment/SLA
work_order.assigned	Service	Technician notification
work_order.completed	Service	Customer notification/report PDF
maintenance.due	Maintenance	Work-order generation/notification
invoice.approved	Finance	PDF generation
invoice.posted	Finance	Email/webhook/AR update
payment.posted	Finance	Receipt/webhook
contract.expiring	CRM	Account-manager/customer notification
document.expiring	Documents	Owner notification

Appendix C - Engineering Rules Checklist

- No direct Prisma access from routes/controllers.
- No cross-module repository imports.
- Use module facades for synchronous domain collaboration.
- Use shared contracts for browser-safe API payloads.
- Never return password hashes, MFA secrets, refresh hashes or internal storage credentials.
- Never trust organizationId from request body as authorization context.
- Every financial and inventory mutation has a transaction plan before implementation.
- Every critical command has an explicit allowed-state transition table.
- Every command produces a stable audit action name.
- Every list endpoint has bounded pagination and allowlisted sort/filter fields.
- Every database migration is reviewed for indexes, nullability, defaults and backfill impact.
- Every uploaded file has size/type/category validation and private access by default.
- Every queue handler is idempotent or safely retryable.
- Every external webhook is signed and delivery attempts are recorded.
- Every module exposes only index.ts/facade public surface to sibling modules.
- Every tenant-owned unique constraint includes organizationId where uniqueness is tenant-local.
- Every posted ledger transaction is corrected with reversal/adjustment, not destructive editing.
- Every production change has health checks and rollback/restore awareness.

Appendix D - Recommended Shared Package Layout

```

shared/src/
├── contracts/
│   ├── auth/
│   ├── customers/
│   ├── vendors/
│   ├── procurement/
│   ├── inventory/
│   ├── projects/
│   ├── assets/
│   ├── service/
│   └── finance/
├── schemas/
├── enums/
├── constants/
├── permissions/
├── utils/
└── index.ts
# pure/browser-safe only

```

Browser-safety rule

JWT signing, password hashing, MinIO credentials, Prisma clients, secret environment access and server-only cryptography never belong in the root shared package.

Appendix E - Final Architecture Statement

NEXORA ERP follows an opinionated domain-first modular monolith architecture using Fastify and TypeScript. Business capabilities are isolated into self-contained modules with explicit public facades; browser-safe API contracts and validation schemas are shared through the TypeScript monorepo. PostgreSQL and Prisma provide transactional persistence, MinIO provides private object storage, Redis and BullMQ handle cache, locks, asynchronous workloads and scheduled operations, and the entire platform is containerized and deployed as one coordinated system until independent scaling requirements justify extraction.

Appendix F - Commercial Completeness Addendum

Recommended missing features, modules, database entities, APIs and roadmap updates | Version 1.1 Addendum

Purpose of this addendum

The original NEXORA ERP specification already defines a strong integrated ERP baseline covering CRM, projects, procurement, inventory, assets, field service, maintenance, finance, portals, documents, audit, security, deployment and testing. This addendum extends the same architecture with practical commercial modules that real ERP clients commonly expect after the first production implementation.

Professional completion note

- The project specification is strong enough to start implementation, but the product should be considered complete only when the end-to-end lifecycle works with permissions, tenant isolation, audit, validations, transactions and tests.
- The additions below should not replace the existing roadmap. They should be added as phased extensions, with Number Sequence, Import Wizard, Tax, Stock Count and Bank Reconciliation receiving higher priority.
- All new tenant-owned tables follow the existing rule: organizationId is mandatory, branchId is added where branch-level operation or reporting is required, and high-risk mutations create audit logs and approval records.

Added module	Business value	Primary owner	Recommended phase
Number Sequence Management	Controls PR, PO, GRN, INV, AST, TKT and other business numbers per tenant/branch/year.	Platform	Phase 0/1
Vendor Onboarding & Risk	Prevents unapproved or risky vendors from being used in procurement and payments.	Procurement	Phase 4.5
Physical Stock Count / Cycle Count	Keeps warehouse balances accurate through periodic counting and variance approvals.	Inventory	Phase 3.5
Landed Cost & Inventory Valuation	Adds freight, customs, handling and insurance into actual inventory/project cost.	Inventory/Finance	Phase 4/9
Tax Engine	Supports configurable sales tax, purchase tax, withholding and tax reports.	Finance	Phase 9
Bank & Cash Management	Adds bank accounts, cash accounts, vouchers, cheque register and reconciliation.	Finance	Phase 9
Purchase Contracts / Blanket POs	Handles long-term supplier rate agreements and repeated purchases.	Procurement	Phase 4.5
Data Import Wizard & Data Quality	Supports real ERP onboarding from Excel/CSV with validation and rollback.	Platform	Phase 2/3
Communication Log	Stores email/SMS/portal communication history for audit and customer/vendor disputes.	Platform/CRM	Phase 10
Technician GPS & Visit Verification	Verifies field visits, site arrival, travel time and service execution proof.	Field Service	Phase 7/11
Custom Report Builder	Allows saved reports, dashboards, scheduled exports and user-specific grid views.	Reports	Phase 10/11
Feature Flags & Module Configuration	Allows modules/features to be enabled per tenant and released safely.	Platform	Phase 1/10
SaaS Subscription & Tenant Billing	Supports NEXORA as a hosted SaaS business with plans, limits and tenant billing.	Platform Commercial	Phase 10/11

Appendix F.1 - Additional Functional Modules

Module	Recommended functionality
1. Number Sequence Management	Define human-readable business numbers per organization, branch, entity type, year and reset policy. Required for purchase requests, RFQs, POs, GRNs, invoices, payments, projects, assets, tickets and work orders. Must be transaction-safe to avoid duplicate numbers under concurrent requests.
2. Vendor Onboarding & Vendor Risk	Create vendor registration requests, verify vendor documents, verify bank details, apply category approvals, calculate risk rating, blacklist vendors when required, and prevent unapproved vendors from RFQ/PO/payment workflows.
3. Physical Stock Count / Cycle Count	Plan warehouse counts, generate count sheets, freeze affected stock, record counted quantity, compare system vs physical stock, require approval for variances, post adjustment entries and preserve count audit history.
4. Landed Cost & Inventory Valuation	Capture freight, customs, insurance, handling, transport and other purchase-related costs. Allocate costs by value, quantity, weight or manual rules. Update cost layers and improve project profitability accuracy.
5. Tax Engine	Configure tax codes, tax rates, taxable parties, sales/purchase tax, withholding tax, exemptions, reverse-charge behavior if needed, and tax reports. Tax calculation must be deterministic and auditable.
6. Bank & Cash Management	Manage bank accounts, cash accounts, receipt vouchers, payment vouchers, cheque register, bank statement imports, matching rules and bank reconciliation. Connect payments to invoices and journal entries.
7. Purchase Contracts / Blanket Purchase Orders	Create long-term supplier agreements with validity period, approved items, agreed rates, maximum quantity/value and release orders. Repeated purchases can be issued against a contract without full RFQ each time.
8. Data Import Wizard & Data Quality	Provide import templates, column mapping, preview, row-level validation, duplicate checks, error download, approval before import, rollback, and import audit. This is essential during real client implementation.
9. Communication Log	Record system emails, SMS, portal messages and manual customer/vendor communications. Store template, recipient, delivery status, related subject, attachments and failure reason.
10. Technician GPS & Visit Verification	Capture work-order check-in, check-out, optional GPS coordinates, travel status, route pings, site-arrival proof, photo proof and customer signature. Enforce privacy and tenant policy settings.
11. Custom Report Builder	Allow users to create saved reports, selected columns, filters, charts, scheduled report emails, dashboard widgets and saved grid views. Permissions must restrict sensitive financial and HR data.
12. Feature Flags & Module Configuration	Enable or disable modules per organization, control beta features, define plan-based access, and maintain configuration history. Useful for SaaS rollout and safe production releases.
13. SaaS Subscription & Tenant Billing	Manage NEXORA tenant plans, user limits, storage limits, feature limits, trial periods, subscription status, subscription invoices, payment status and usage metrics. This supports the software vendor business model.

Appendix F.2 - Database Entity Additions

The following tables extend the existing entity catalog. Each tenant-owned table must include organizationId. Branch-owned operational tables should also include branchId. All financial, inventory and approval-impacting records require audit events.

Module	New tables / entities	Important fields and relationships
Number Sequence	NumberSequence NumberSequenceReservation	organizationId, branchId, entityType, prefix, fiscalYear, currentNumber, padding, resetPolicy, lockedAt. Reservation protects concurrent number generation.
Vendor Onboarding & Risk	VendorOnboardingRequest VendorDocument VendorBankAccount VendorRiskAssessment VendorBlacklist VendorCategoryApproval	Request links to Vendor and ApprovalRequest. Documents link to Document. Bank account verification must be audited. Blacklisted vendors cannot be selected for PO/payment without override permission.
Purchase Contracts / Blanket PO	PurchaseContract PurchaseContractItem BlanketPurchaseOrder BlanketPurchaseOrderItem PurchaseReleaseOrder PurchaseReleaseOrderItem	Contract links to Vendor, Product and optional ApprovalRequest. Release orders consume contract quantity/value and may create PO lines.
Stock Count / Cycle Count	StockCount StockCountLine StockCountVariance StockCountApproval StockCountPosting	StockCount links to Warehouse/Location. Lines store systemQty, countedQty and varianceQty. Posting creates StockTransaction entries after approval.
Landed Cost / Valuation	LandedCost LandedCostLine LandedCostAllocation InventoryCostLayer	LandedCost links to PO/GRN/SupplierInvoice. Allocation links cost to products/receipt lines. CostLayer supports weighted average/FIFO policy if adopted.
Tax Engine	TaxCode TaxRate TaxRule TaxJurisdiction TaxTransaction WithholdingTaxRule	TaxRule is applied during quotation, invoice, purchase and expense calculation. TaxTransaction stores calculated tax for audit and reporting.

Module	New tables / entities	Important fields and relationships
Bank & Cash Management	BankAccount CashAccount BankStatement BankStatementLine BankReconciliation PaymentVoucher ReceiptVoucher ChequeRegister	Bank/cash accounts link to Account in chart of accounts. Statement lines are matched to Payment, JournalEntry or voucher records.
Data Import & Quality	ImportTemplate ImportBatch ImportMapping ImportRow ImportRowError DuplicateCheckRule	ImportBatch stores status, subjectType, uploaded document and validation summary. Imported records should reference source batch for traceability.
Communication Log	CommunicationTemplate CommunicationLog EmailDeliveryLog SmsDeliveryLog MessageAttachment	Logs link to subjectType/subjectId, recipient, channel, template, delivery status and Document attachments.
Technician GPS / Visit	ServiceVisit ServiceVisitLocation TechnicianLocationPing TechnicianRoute WorkOrderCheckIn WorkOrderCheckOut	ServiceVisit links to WorkOrder and technician Employee. Location data should be retained according to organization privacy policy.
Custom Reporting	ReportTemplate SavedReport ScheduledReport ReportExecution DashboardWidget UserDashboard SavedView	Saved reports must store permission scope and filter JSON. Report executions store export Document and job status.

Module	New tables / entities	Important fields and relationships
Feature Flags / SaaS Billing	FeatureFlag OrganizationFeature ModuleConfiguration SystemConfigurationHistory SaaSPlan SaaSSubscription SaaSSubscriptionFeature SaaSInvoice TenantUsageMetric TenantStorageUsage	OrganizationFeature controls enabled modules. SaaSSubscription links plan limits to organization. Usage metrics support billing and limit enforcement.

Appendix F.3 - Workflow Integration Rules

Workflow	Atomic transaction must include	Async after commit
Generate business number	NumberSequence row lock/reservation + assigned business number + target entity creation.	None, except audit/reporting event.
Vendor onboarding approval	Vendor request + approval action + vendor status/category approval + audit.	Notification to requester/procurement.
Physical stock count posting	Approved count + variance lines + stock adjustment + immutable stock ledger + audit.	Stock count report PDF/export.
Landed cost posting	Landed cost header/lines + cost allocation + inventory cost layer/value update + audit.	Recalculated project profitability snapshot.
Tax calculation on invoice	Invoice line totals + TaxTransaction records + invoice totals + audit.	Tax report cache/update job.
Bank reconciliation close	Matched statement lines + reconciliation status + journal/payment links + audit.	Reconciliation report PDF.
Purchase release order	Contract balance check + release order + PO/PO lines + audit.	Vendor notification.
Import batch commit	Validated rows + created/updated target records + import result + audit.	Import completion notification.
Technician visit completion	Work order status + visit timestamps/location proof + service report + parts consumption + audit.	Customer report PDF and notification.
Scheduled report execution	ReportExecution status + generated Document metadata when complete.	Email delivery and dashboard refresh.

Rule

Do not process stock variance, landed cost posting, tax posting, payment posting, bank reconciliation closure or approval state asynchronously when the financial or inventory effect must be consistent immediately. Use queues only for documents, emails, notifications, exports, analytics and external webhooks.

Appendix F.4 - API Endpoint Additions

Area	Endpoint examples	Notes
Number Sequence	GET /api/v1/number-sequences POST /api/v1/number-sequences POST /api/v1/number-sequences/:id/reset	Restricted platform/admin permission. Use internal service for issuing numbers, not direct frontend calls.
Vendor Onboarding	POST /api/v1/vendor-onboarding/requests POST /api/v1/vendor-onboarding/:id/submit POST /api/v1/vendor-onboarding/:id/approve POST /api/v1/vendors/:id/blacklist	Approval and maker-checker rules apply.
Stock Count	POST /api/v1/stock-counts POST /api/v1/stock-counts/:id/start POST /api/v1/stock-counts/:id/submit POST /api/v1/stock-counts/:id/post	Posting creates stock ledger entries.
Landed Cost	POST /api/v1/landed-costs POST /api/v1/landed-costs/:id/allocate POST /api/v1/landed-costs/:id/post	Posting should be idempotent.
Tax Engine	GET /api/v1/tax-codes POST /api/v1/tax-rules POST /api/v1/tax/calculate GET /api/v1/tax/reports	Calculation endpoint returns deterministic preview before invoice posting.

Area	Endpoint examples	Notes
Bank & Cash	GET /api/v1/bank-accounts POST /api/v1/bank-statements/import POST /api/v1/bank-reconciliations/:id/close POST /api/v1/vouchers/payment	Reconciliation close creates audit and optional journal links.

Appendix F.4 - API Endpoint Additions continued

Area	Endpoint examples	Notes
Purchase Contracts	POST /api/v1/purchase-contracts POST /api/v1/purchase-contracts/:id/approve POST /api/v1/purchase-contracts/:id/create-release-order	Release order validates remaining contract quantity/value.
Data Import	POST /api/v1/imports/upload POST /api/v1/imports/:id/validate POST /api/v1/imports/:id/commit POST /api/v1/imports/:id/rollback	Commit must be transactional by batch or controlled chunk.
Communication Log	GET /api/v1/communications POST /api/v1/communications/send GET /api/v1/communications/:id/delivery	Attach to customer, vendor, ticket, invoice or project timeline.
Technician Visit	POST /api/v1/work-orders/:id/check-in POST /api/v1/work-orders/:id/location POST /api/v1/work-orders/:id/check-out	Technician-only scope, privacy policy enforced.
Report Builder	POST /api/v1/report-templates POST /api/v1/saved-reports POST /api/v1/scheduled-reports GET /api/v1/report-executions/:id	Every saved report stores permission scope and selected fields.
Feature Flags	GET /api/v1/features POST /api/v1/organization-features PATCH /api/v1/module-configurations/:id	Useful for per-tenant module enablement and beta rollout.
SaaS Billing	GET /api/v1/saas/plans POST /api/v1/saas/subscriptions GET /api/v1/saas/usage POST /api/v1/saas/invoices/:id/post	Used by NEXORA platform owner, not normal tenant users.

Permission key additions

Permission key	Purpose
number_sequence.manage	Manage business number formats and reset rules.
vendor.onboard	Create and submit vendor onboarding requests.
vendor.risk.manage	Assess vendor risk, blacklist vendors and approve vendor categories.
stock_count.manage	Create and manage physical/cycle counts.
stock_count.post	Approve/post stock count variances.
landed_cost.manage	Create, allocate and post landed costs.
tax.manage	Manage tax codes, tax rates and tax rules.
bank.manage	Manage bank/cash accounts and reconciliation.
purchase_contract.manage	Manage purchase contracts and blanket POs.
import.manage	Validate, commit and rollback data imports.
communication.view	View communication history.
communication.send	Send approved communications.
technician.location.manage	View/manage technician visit location proof.
report_builder.manage	Create custom reports, scheduled reports and dashboard widgets.
feature.manage	Manage tenant feature flags and module configuration.
saas.manage	Manage tenant subscriptions, plans and platform billing.

Appendix F.5 - Updated Roadmap Placement

Phase	Existing theme	Add these features to make the product commercially complete
Phase 0	Repository foundation	NumberSequence service, feature flag seed model, module registry seed, base report export pattern.
Phase 1	Identity & organization	OrganizationFeature, ModuleConfiguration, SaaS plan guard, number format setup, admin configuration history.
Phase 2	Business masters	Import templates for employees, customers, vendors, products and warehouses. Vendor onboarding request baseline.
Phase 3	Inventory	Stock count, cycle count, stock freeze, variance approval and inventory cost layer baseline.
Phase 4	Procurement	Vendor risk, purchase contracts, blanket POs, release orders and landed-cost capture from GRN/supplier invoice.
Phase 5	Projects	Project cost recalculation using landed cost, committed cost and actual cost layers.
Phase 6	Assets	Asset cost history, warranty document expiry and QR access audit improvements.
Phase 7	Field service	Technician visit check-in/check-out, optional GPS proof, route events and service visit record.
Phase 8	Maintenance	Maintenance cost by asset, recurring service package entitlement consumption.
Phase 9	Finance	Tax engine, bank/cash management, vouchers, cheque register, bank statement import and reconciliation.
Phase 10	Enterprise controls	Communication log, custom report builder, scheduled reports, saved views, feature/module configuration.
Phase 11	Advanced operations	SaaS subscription billing, tenant usage metrics, advanced analytics and offline PWA synchronization policies.

MVP priority recommendation

Priority	Must-have additions before serious client pilot	Reason
P0	Number Sequence Management	Every ERP needs clean and controlled business numbers.
P0	Data Import Wizard baseline	Real clients usually start with Excel/CSV data migration.
P1	Stock Count / Cycle Count	Inventory cannot stay trusted without physical verification.
P1	Tax Engine baseline	Invoices, purchases and reports require controlled tax calculation.
P1	Bank/Cash Management baseline	Payments need reconciliation and voucher-level control.
P2	Vendor Onboarding & Risk	Improves procurement governance and fraud control.
P2	Landed Cost	Improves real profitability and inventory valuation.
P3	Report Builder, GPS visit proof, SaaS billing	Important for commercial maturity but can follow after core pilot.

Appendix F.6 - Updated Definition of Done

The existing Definition of Done remains valid. For the new modules, the following checks must also be satisfied before a module is called complete.

Area	Additional Definition of Done
Number sequences	Concurrent requests cannot generate duplicate numbers. Reset rules are tested by fiscal year, branch and organization.
Vendor onboarding	Unapproved or blacklisted vendors cannot be used in RFQ, PO or payment workflows without explicit override permission and audit.
Stock count	Variance posting creates immutable stock ledger entries and cannot bypass approval thresholds.
Landed cost	Cost allocation reconciles to total landed cost and updates inventory/project costing consistently.
Tax engine	Tax calculation is repeatable, auditable and stored at transaction time so later tax rule changes do not rewrite history.
Bank reconciliation	Closed reconciliations cannot be silently edited; corrections require reversal or reopening with permission and audit.
Data import	Failed rows are explainable, valid rows can be previewed, committed records link back to import batch, and rollback policy is defined.
Communication log	Delivery status, failure reason and related business object are traceable without exposing secrets or credentials.
Technician GPS	Location collection is configurable by tenant policy and sensitive location data has retention/access controls.
Report builder	Saved reports cannot bypass RBAC, tenant isolation, branch scope or sensitive field restrictions.
Feature flags	Disabled modules are hidden in UI and blocked at API/service level. Configuration changes are audited.
SaaS billing	Plan limits are enforced consistently for users, storage, modules and usage metrics.

Final architecture impact statement

These additions do not require changing the approved architecture. They fit naturally inside the existing TypeScript monorepo, Fastify modular monolith, PostgreSQL/Prisma persistence layer, MinIO document storage, Redis/BullMQ job processing and domain-event model. They should be implemented as additional modules with strict service/repository/facade boundaries and the same tenant, permission, audit and transaction rules already defined in the main specification.

Appendix G - Frontend Implementation Completion Addendum

Reusable component system, forms, data grids, screen-by-screen UI rules, route-handler policy, offline sync endpoint and shell consistency | Version 1.2 | 08 September 2026

Purpose of this addendum

This appendix extends the existing NEXORA ERP specification with the missing frontend implementation detail found during source review. It is additive: it does not replace the locked Fastify, TypeScript, PostgreSQL, Prisma, MinIO, Redis, BullMQ and Next.js architecture already approved for the platform.

The source review found that backend endpoint coverage is strong, but the frontend still needs a complete reusable component system, complete module CRUD forms, page-by-page UI specifications, full create/edit/detail workflows, a documented Next.js route-handler policy, the missing backend route for technician offline sync, and consistent application shell wrapping across all authenticated pages.

G.1 Scope being closed

Gap from source review	Required completion decision
Reusable frontend component system	Use shadcn/ui as the base primitive component system, wrapped by NEXORA-owned app, form and data-grid components.
Complete CRUD forms for each module	Use React Hook Form plus Zod resolver for all create/edit/command forms. Every editable aggregate requires create, edit, details and workflow command UI where applicable.
Proper screen-by-screen UI implementation	Every route must have a UI contract: purpose, permissions, table columns, filters, form fields, actions, dialogs, API calls, states and responsive behavior.
Complete frontend create/edit/detail workflows	Each module must implement list, create, detail, edit and allowed command transitions, not only read-only dashboard/workbench pages.
Next.js API routes	Do not create a second business API. Document a constrained Next.js route-handler/BFF policy for frontend-only utilities while ERP domain APIs remain Fastify /api/v1.
Missing /portal/technician/offline-sync backend route	Add a Fastify POST /api/v1/portal/technician/offline-sync endpoint with technician scope, idempotency, conflict handling and atomic business updates.
Inconsistent frontend shell wrapping	Move shell enforcement to Next.js route-group layouts so all authenticated ERP, portal and technician pages are wrapped consistently.

G.2 Locked frontend technology decisions

Concern	Locked decision	Implementation rule
Primitive UI components	shadcn/ui	Use generated shadcn/ui components under frontend/src/components/ui. Customize through Tailwind tokens and NEXORA wrappers, not one-off per page styling.
Forms	React Hook Form	All create/edit/command forms use useForm, FormProvider and typed submit handlers. Avoid untyped local state for canonical form fields.
Validation	Zod shared contracts	Use shared browser-safe Zod contracts with @hookform/resolvers/zod. Backend may apply stricter server-only validation after structural validation passes.
Tables and grids	TanStack Table	All ERP list screens use TanStack Table for columns, row models, sorting state, selection, visibility and row actions.
Server state	TanStack Query	All frontend API reads/mutations use centralized query/mutation hooks with invalidation, optimistic rules and error mapping.
Styling	Tailwind CSS	Use design tokens and reusable variants. Page modules must not invent separate visual systems.
API source	Fastify /api/v1	Business data is retrieved from the Fastify backend. Next.js route handlers cannot own ERP domain logic.

G.3 Frontend package and folder layout

The frontend keeps the existing monorepo boundary and introduces a clear component ownership model. Shared UI primitives are allowed in common component folders; business-specific components remain in their owning module.

```
frontend/src/
├── app/
│   ├── layout.tsx
│   ├── (auth)/layout.tsx
│   ├── (erp)/layout.tsx
│   ├── (portal)/layout.tsx
│   └── (technician)/layout.tsx
├── components/
│   ├── ui/
│   ├── app/
│   ├── data/
│   ├── forms/
│   ├── feedback/
│   └── workflow/
├── modules/
│   ├── <domain>/
│   │   ├── api.ts
│   │   ├── columns.tsx
│   │   ├── schemas.ts
│   │   ├── components/
│   │   ├── forms/
│   │   └── pages/
└── lib/
    ├── api-client.ts
    ├── query-client.ts
    ├── permissions.ts
    └── route-map.ts

# Providers only
# Public/auth layout
# AppShell + AuthGuard + TenantGuard
# Customer/vendor portal shell
# Technician PWA shell + OfflineProvider

# shadcn/ui generated primitives
# AppShell, PageHeader, guards, breadcrumbs
# DataTable, toolbar, pagination, filters
# Form wrappers and domain-neutral fields
# EmptyState, ErrorState, LoadingState
# StatusBadge, ApprovalTimeline, AuditTimeline

# endpoint constants + query/mutation functions
# TanStack Table column definitions
# UI-local schema composition if required
# module-specific UI
# create/edit/command forms
# route-level containers when needed
```

G.4 Reusable component system using shadcn/ui

shadcn/ui is the approved source for reusable primitive components. These components are copied into the codebase and become project-owned source. They must be centralized, themed and reused across all modules.

Component layer	Examples	Rules
Primitive UI	Button, Input, Textarea, Select, Checkbox, RadioGroup, Dialog, Sheet, Drawer, Popover, Tooltip, Tabs, Card, Badge, Alert, DropdownMenu, Table, Skeleton, Toast	Lives in components/ui. Keep primitive and reusable. Do not add procurement/finance/project business logic here.
Application layout	AppShell, PortalShell, TechnicianShell, Sidebar, Topbar, Breadcrumbs, PageHeader, ModuleHeader, TenantSwitcher	Lives in components/app. Authenticated pages use shells through route-group layout, not ad hoc per page.
Data grid	DataTable, DataToolbar, DataPagination, ColumnVisibilityMenu, BulkActionBar, RowActionsMenu, FilterPanel	Built using TanStack Table and shadcn primitives. All ERP list pages use this layer.
Forms	FormSection, FormFieldWrapper, MoneyInput, QuantityInput, DatePickerField, FileUploadField, StatusSelect, TenantCombobox, UserPicker, ProductPicker, WarehousePicker	Built on React Hook Form. Components receive control/name and never own final business validation.
Workflow and audit	StatusBadge, ApprovalTimeline, StateTransitionPanel, AuditTimeline, ActivityTimeline, PermissionGate, MakerCheckerNotice	Used on detail pages and command dialogs. Must reflect canonical status and permission catalogs.
Feedback states	LoadingState, EmptyState, ErrorState, ForbiddenState, NotFoundState, ValidationSummary, ConflictState	Every page and mutation must show consistent state handling.

G.5 Form standard - React Hook Form plus Zod

Every create, edit and workflow command form must use React Hook Form. Shared Zod contracts provide browser-safe structural validation and inferred TypeScript types. Backend validation remains authoritative for permission, tenant, branch, ownership, state transition and transaction rules.

- Use useForm with zodResolver(sharedSchema) for all module forms.
- Wrap complex forms in FormProvider so nested field components can consume form context.

- Use Controller for shadcn Select, DatePicker, Combobox, file picker and other controlled components.
- Create forms must submit through TanStack Query mutations, then invalidate the relevant list/detail/timeline queries.
- Edit forms must only expose editable fields. Status fields are not freely patchable and must use command dialogs/endpoints.
- Command forms such as submit, approve, reject, receive, post, cancel, close and pay must show current status, required permission, irreversible effects and validation errors.
- Financial and inventory forms must disable duplicate submission while pending and use idempotency keys where the backend requires them.
- API validation errors must be mapped to field errors when possible and to ValidationSummary when they are record-level errors.

```
const form = useForm<CreatePurchaseRequestInput>({
  resolver: zodResolver(CreatePurchaseRequestSchema),
  defaultValues: { projectId: undefined, requiredDate: '', reason: '', items: [] }
});

const mutation = useCreatePurchaseRequestMutation({
  onSuccess: () => queryClient.invalidateQueries({ queryKey: purchaseRequestKeys.lists() })
});

function onSubmit(values: CreatePurchaseRequestInput) {
  mutation.mutate(values);
}
```

G.6 Table and grid standard - TanStack Table

All ERP grid/list screens must use TanStack Table. shadcn/ui table markup may be used for rendering, but table behavior, column definitions, sorting state, selection state, pagination state and row actions are controlled by TanStack Table.

Table capability	Requirement
Columns	Each module defines typed columns in modules/columns.tsx. Column ids must be stable so saved views and report builder features can depend on them.
Pagination	Server-side page/pageSize for ERP grids. Cursor pagination only for timelines/events where appropriate. No unbounded fetch-all lists.
Sorting/filtering	Frontend exposes only allowlisted sort and filter fields supported by the backend endpoint. Raw database expressions are forbidden.
Row actions	Actions are permission-gated and status-aware. Example: Approve appears only when current user can act and the record is in an approvable state.
Bulk actions	Allowed only where business rules support batch operations safely. Critical financial/inventory mutations require explicit review dialogs.
Loading/empty/error	Use Skeleton, EmptyState and ErrorState consistently. Empty states should explain the next allowed action.
Export	Large export uses report export API/job, not browser-side CSV generation from partial grid rows.

G.7 Complete CRUD and command workflow requirement

Every module must move beyond read-only lists. For each editable aggregate, the frontend must provide a complete workflow surface using the backend command endpoints and canonical statuses.

UI surface	Required behavior
List page	Paginated/filterable TanStack Table, search/filter panel, permission-aware create button, status badges, row actions, empty/error/loading states.
Create page/dialog	React Hook Form, Zod resolver, server-side duplicate/status validation, submit mutation, success navigation or detail refresh.
Detail page	Header with business number/status, key facts, related tabs, activity timeline, audit timeline, documents, comments and allowed command panel.
Edit page/dialog	Editable fields only. No free status PATCH. Changes are audited by backend.
Command dialogs	Submit, approve, reject, return, publish, close, receive, inspect, post, pay, cancel, install, replace, retire, assign and complete actions use explicit dialogs and command endpoints.
Permission denial	Hide unavailable actions and show ForbiddenState for direct URL access. Permissions alone are not enough; backend remains authoritative.

G.8 Screen-by-screen UI specification template

Every frontend route must have an implementation contract before it is called complete. This contract can be stored beside the module source or in docs/frontend-screens, but it must be enforced by review.

Screen contract field	Required content
Route and owner	Next.js route path, owning module, page file, route group and shell.
Purpose	The business question the screen answers and the primary user action.
Permissions	Required view permission plus action-level permissions for buttons, menu items and command dialogs.
API mapping	All GET/POST/PATCH/PUT/DELETE/command endpoints used by the screen, query keys and invalidation rules.
Data table spec	Columns, hidden columns, filters, sort fields, pagination mode, row actions and bulk actions.
Form spec	Fields, required/optional status, defaults, validation schema, server error mapping and command idempotency needs.
State model	Loading, empty, forbidden, not found, validation error, conflict, stale data and offline state where applicable.
Responsive behavior	Desktop table, tablet cards or condensed layout, mobile action sheet and sticky submit/action areas.
Audit/traceability	Activity timeline, status history, audit log entry points and document links when sensitive.

G.9 Required screen families by module

Module area	Required frontend screens
Identity and organization	Users, sessions, roles, permissions, branches, departments, teams, organization settings, number sequences, feature/module configuration.
HR	Employees list/create/detail/edit, attendance grid, leave requests create/submit/approve/cancel, payroll run create/calculate/approve/post.
CRM and sales	Leads, opportunities, site surveys, quotations, contracts, customers, customer sites, timeline and conversion command screens.
Vendors and procurement	Vendors, onboarding/risk, purchase requests, approvals, RFQs, supplier quotations, comparison, purchase orders, GRNs, quality inspection, purchase contracts and landed cost capture.
Inventory	Products, categories, warehouses, locations, stock balances, ledger, serial lookup, reservations, transfers, adjustments, stock count/cycle count.
Projects	Projects, phases, tasks, milestones, teams, sites, budgets, BOM, material request, costing, risks, issues, handover and timeline.
Assets	Asset list/detail, register from stock, install, replace, retire, QR rotate, warranty, RMA and asset history.
Service and maintenance	Tickets, work orders, technician assignment, service reports, parts usage, maintenance plans, schedules, executions and technician PWA screens.
Finance	Customer invoices, supplier invoices, three-way match, payments, expenses, accounts, journal entries, tax, bank/cash, vouchers and reconciliation.
Platform	Documents, upload intent/complete upload, reports/exports, custom report builder, communications, notifications, audit logs, global search, calendar and SaaS billing.

G.10 API consumption and frontend route-handler policy

NEXORA must not split business logic between Next.js API routes and the Fastify backend. The approved business API remains Fastify under /api/v1. Next.js route handlers, if implemented, are only a constrained presentation-layer/BFF utility and must never own ERP domain rules, Prisma access, MinIO credentials, Redis queues, approval logic, stock logic or finance logic.

Area	Allowed	Forbidden
Business API	Call Fastify /api/v1 through centralized frontend api-client and module api.ts files.	Do not create Next.js /api/customers, /api/purchase-orders, /api/invoices or other duplicate business APIs.
Next.js route handlers	Only frontend utilities such as /api/frontend/runtime-config, /api/frontend/health or a narrow BFF proxy approved by architecture review.	No Prisma client, no MinIO SDK, no password hashing, no secret access, no direct domain mutation and no RBAC bypass.
Frontend data access	TanStack Query hooks and typed API functions. Query keys are centralized per module.	Raw fetch calls inside page.tsx or component bodies are forbidden except through the approved client wrapper.
Contracts	Import shared browser-safe contracts and enums. Server-only extensions stay in backend.	Do not import backend, database, worker or server-only shared files into frontend.

Allowed frontend route handlers, if needed:

```
GET /api/frontend/runtime-config
GET /api/frontend/health
POST /api/frontend/bff/* # optional, architecture-approved proxy only
```

Primary ERP business API remains:
 /api/v1/auth/*
 /api/v1/customers
 /api/v1/purchase-requests
 /api/v1/purchase-orders
 /api/v1/goods-receipts
 /api/v1/invoices
 /api/v1/work-orders
 ... all domain endpoints implemented in Fastify.

G.11 Backend endpoint addition - technician offline sync

The frontend and shared portal contracts reference technician offline sync. The backend must expose a matching Fastify route so the PWA can submit offline commands safely when connectivity returns.

Method	Endpoint	Purpose	Permission/scope	Notes
POST	/api/v1/portal/technician/offline-sync	Synchronize technician PWA offline work-order commands, evidence references, checklist updates, status updates, service-report data and parts usage.	Authenticated technician portal scope; assigned technician only; tenant and branch enforced.	Requires Idempotency-Key or clientBatchId/clientCommandId dedupe. Reject wrong tenant, wrong technician, stale commands and conflicting payload replays.

POST /api/v1/portal/technician/offline-sync
 Request:

```
{
  "deviceId": "tech-device-001",
  "clientBatchId": "batch-2026-09-08-001",
  "technicianEmployeeId": "uuid",
  "tenantClockAt": "2026-09-08T12:00:00Z",
  "commands": [
    {
      "clientCommandId": "cmd-001",
      "workOrderId": "uuid",
      "type": "STATUS_CHANGE",
      "occurredAt": "2026-09-08T11:45:00Z",
      "payload": { "nextStatus": "ON_SITE" }
    }
  ]
}
```

Response:

```
{
  "data": {
    "clientBatchId": "batch-2026-09-08-001",
    "accepted": ["cmd-001"],
    "replayed": [],
    "rejected": []
  },
  "meta": { "requestId": "req_..." }
}
```

- Offline command application must be idempotent per organizationId, deviceId and clientCommandId.
- Commands that change work-order status, service report state, parts consumption or asset history must be committed in one PostgreSQL transaction with audit records.
- Photos/signatures/files should use the existing document upload-intent and complete-upload design; offline sync should reference completed document ids or pending evidence records, not raw large file blobs when avoidable.
- Queue workers may generate PDFs, notifications and analytics after commit, but cannot be the source of truth for stock, money, approval state or work-order critical status changes.

G.12 Consistent AppShell and portal shell wrapping

Authenticated pages must be wrapped by layout-level shells instead of relying on each page to import AppShell manually. This prevents pages from opening without sidebar, auth guard, tenant context, navigation, breadcrumbs or notification context.

Route group	Shell/guard	Applies to
app/(auth)	AuthLayout only	Login, MFA, password reset and other public identity screens. Must not show ERP sidebar.
app/(erp)	AppShell + AuthGuard + TenantGuard + PermissionContext	Main ERP modules: dashboard, CRM, projects, procurement, inventory, assets, service, finance, HR, vendors, reports, documents, approvals, audit and administration.
app/(portal)	PortalShell + PortalGuard	Customer and vendor portal dashboards, projects, assets, tickets, invoices, payments, documents and approvals.
app/(technician)	TechnicianPwaShell + TechnicianGuard + OfflineProvider	Technician jobs, work orders, QR scanning, offline queue, evidence capture and offline sync screens.
app/layout.tsx	Providers only	Theme, QueryClientProvider, toast provider, global error boundary and shared context. No sidebar decision here.

- No authenticated page.tsx should import AppShell directly once route-group layouts are in place.
- Public/auth pages must never be rendered inside the ERP shell.
- Customer/vendor portal pages must not show internal ERP navigation.
- Technician PWA pages must support mobile-first layout, offline queue visibility and sync status.
- Navigation registry must remain permission-aware and module-aware; disabled modules are hidden in UI and blocked at API/service level.

G.13 Frontend API and query-key conventions

Concern	Required convention
API client	All requests go through frontend/src/lib/api-client.ts or modules/core/api-client.ts. The client injects auth, organization context, requestId handling and stable error parsing.
Module API file	Every module owns modules/api.ts with endpoint constants, typed request/response functions and query-key factories.
Query keys	Use stable factories such as purchaseRequestKeys.list(filters), purchaseRequestKeys.detail(id), projectKeys.timeline(id).
Mutations	Command mutations define invalidation targets and must not silently mutate critical cached financial/inventory values without refetch.
Errors	Map backend error.code and details into field errors, conflict notices or forbidden/not-found states.
Permissions	Use shared permission constants for UI gating, but never rely on frontend checks as security. Backend remains authoritative.

G.14 Module CRUD implementation matrix

The following matrix is the minimum frontend delivery contract. Each row requires list, create, detail, edit where editable, and command dialogs where a state transition exists.

Area	Create/edit forms	Detail/workflow screens
Organization and RBAC	Organization settings, branches, departments, teams, users, roles and permissions.	Membership detail, session history, role-permission matrix, audit events and feature/module settings.
Masters	Employees, customers, customer sites, vendors, products, warehouses and import batches.	Profile/detail pages, documents, contacts, related transactions, history and status changes.
Procurement	Purchase request, RFQ, supplier quotation, PO, GRN, inspection, purchase contract and landed cost forms.	Approval, comparison, vendor selection, PO submit/approve/send/cancel, receive goods, inspect and three-way match links.
Inventory	Product, warehouse, transfer, reservation, adjustment, stock count and batch/serial forms.	Stock balance, immutable ledger, serial history, transfer dispatch/receive and adjustment/variance posting.
Projects and assets	Project, task, milestone, BOM, budget, material request, asset registration, installation, warranty and RMA forms.	Project timeline, costing, handover, asset lifecycle, QR actions, install/replace/retire and warranty/RMA history.
Service and maintenance	Ticket, work order, assignment, service report, parts used, maintenance plan and schedule forms.	Technician workflow, SLA tracking, route/status commands, completion report, customer confirmation and offline sync queue.

Area	Create/edit forms	Detail/workflow screens
Finance	Customer invoice, supplier invoice, payment, expense, account, journal, tax, voucher and reconciliation forms.	Invoice approval/post/send/cancel, payment allocation, AP/AR aging, journal posting, three-way match and bank reconciliation close.
Platform and portals	Document upload, report export, saved report, communication template/log, SaaS plan/subscription and feature settings.	Customer/vendor portal detail pages, report executions, delivery logs, audit trails, usage and billing screens.

G.15 Updated frontend Definition of Done

- A frontend page is not done when it renders static data; it is done when its route, shell, permissions, API mapping, loading, empty, error, forbidden, not-found and responsive states are complete.
- A form is not done when fields appear; it is done when React Hook Form, Zod validation, defaults, server error mapping, success handling, invalidation, idempotency where required and tests are complete.
- A grid is not done when rows display; it is done when TanStack Table columns, filters, sorting, pagination, row actions, bulk rules, loading/empty/error states and permission gates are complete.
- A workflow is not done when a button exists; it is done when allowed-state transitions, confirmation dialog, backend command endpoint, audit behavior, cache invalidation and forbidden/conflict handling are complete.
- A module is not frontend-complete until list, create, detail, edit and all approved command workflows exist for its aggregate roots.
- All business API calls must pass through centralized API/query layers; raw fetch inside screens is a review blocker.
- Every authenticated page must be reachable inside the correct shell and must not duplicate layout/sidebar logic locally.
- Frontend implementation must include unit/component tests for reusable components and critical forms, integration tests for API hooks, and E2E tests for the most important workflows.

G.16 Updated roadmap placement

Phase	Frontend additions
Phase 0 - Repository foundation	Install/configure shadcn/ui, React Hook Form, TanStack Table, hookform resolvers, component folders, API client conventions, QueryClient setup and route-group shell foundation.
Phase 1 - Identity and organization	Implement AppShell/AuthGuard/TenantGuard, RBAC-aware navigation, users/roles/branches/departments CRUD forms and permission matrix UI.
Phase 2 - Business masters	Implement complete CRUD forms/tables/detail pages for employees, customers, customer sites, vendors, products and warehouses; add import wizard UI baseline.
Phase 3 - Inventory	Implement inventory grids, stock ledger, reservations, transfers, adjustments, serial lookup and stock count/cycle count forms.
Phase 4 - Procurement	Implement end-to-end procurement screens from PR through approval, RFQ, quotation comparison, supplier selection, PO, GRN, inspection and landed cost capture.
Phase 5 to Phase 9	Implement projects, assets, field service, maintenance and finance frontend workflows with complete create/edit/detail/command screens.
Phase 10 to Phase 11	Implement report builder, saved views, customer/vendor portals, technician PWA offline sync, communication log, feature flags, SaaS billing and advanced responsive/offline behavior.

G.17 Acceptance checklist for closing the frontend gaps

Checkpoint	Pass condition
Reusable components	components/ui contains shadcn primitives and components/app, components/data, components/forms and components/feedback contain NEXORA wrappers used by modules.
Forms	Each editable module uses React Hook Form with Zod resolver and typed submit handlers. grep for unapproved raw form patterns should be part of CI review.
Tables	All ERP grids use TanStack Table. Raw hand-built tables are allowed only for small read-only detail summaries.
Screen contracts	Every page has route owner, permission, API, table, form, action, state and responsive behavior documented.
Workflows	State transitions use explicit command endpoints and dialogs; status is never freely PATCHed from generic edit forms.
Next route handlers	Any frontend/src/app/api route is architecture-reviewed and contains no domain persistence/business logic.
Offline sync	POST /api/v1/portal/technician/offline-sync exists in backend routes, has contract tests, idempotency tests, tenant/technician scope tests and frontend mutation coverage.
Shell wrapping	All authenticated internal routes live under the ERP route group and receive AppShell automatically; portals and technician PWA receive their own shells.

Checkpoint	Pass condition
CI gates	typecheck, lint, route coverage, frontend component/form/table checks and critical E2E workflows must pass before marking frontend complete.

Final addendum impact statement

These frontend additions strengthen the existing architecture without changing it. shadcn/ui, React Hook Form and TanStack Table fit inside the approved Next.js TypeScript frontend. The missing technician offline sync route belongs in the Fastify backend /api/v1 surface. Next.js route handlers are documented as a constrained presentation-layer option, not a second ERP API. All additions preserve tenant isolation, RBAC, auditability, transaction boundaries, shared contracts and the domain-first modular-monolith structure.